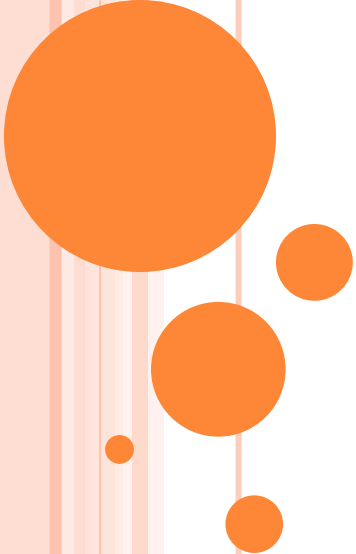




CSE 205: DIGITAL LOGIC DESIGN

Dr. Tanzima Hashem

Professor

CSE, BUET

TEXTBOOK

- Digital Design (5th Edition)
 - M. Morris Mano
 - Michael D. Ciletti



DIGITAL SYSTEMS

- Digital Systems
 - Example: digital cameras, digital telephones, digital television, digital computers...
 - Used in communication, business transactions, traffic control, space guidance, medical treatment, weather monitoring, the Internet ...
- Why study digital logic design
 - To design and build a digital system



BOOLEAN (BINARY) LOGIC

- Deals with binary variables and binary logic functions
- Binary variables has two discrete values
 - 0, False, Open
 - 1, True, Close
- Three basic logical operations
 - AND (.); OR (+); NOT (')



TRUTH TABLE

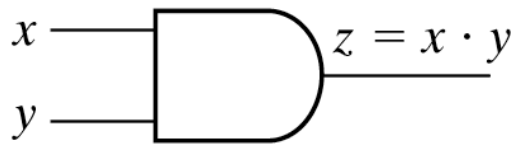
- Specifies results for all possible input combinations

<i>Truth Tables of Logical Operations</i>					
AND			OR		
<i>x</i>	<i>y</i>	$x \cdot y$	<i>x</i>	<i>y</i>	$x + y$
0	0	0	0	0	0
0	1	0	0	1	1
1	0	0	1	0	1
1	1	1	1	1	1
			<i>x</i>		x'
			0		1
			1		0

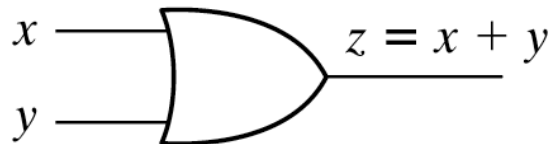


LOGIC GATES

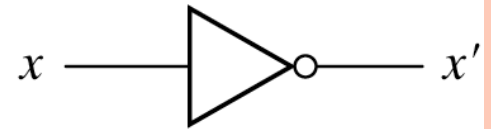
- Logic gates are electronic circuits that operate on one or more input signals to produce an output signal



(a) Two-input AND gate



(b) Two-input OR gate



(c) NOT gate or inverter

Fig. 1-4 Symbols for digital logic circuits



BOOLEAN FUNCTION

- An algebraic expression
 - Binary variables
 - Constants 0 and 1
 - Logic operation symbols

$$f(x,y,z) = (x + y')z + x'$$

- Some terminology, notation and precedence:
 - f is the name of the function.
 - (x,y,z) are the input variables, each representing 1 or 0.
 - A literal is a single variable within a term, in complemented or uncomplemented form. The function above has four literals: x , y' , z , and x' .
 - Parenthesis has the highest precedence, followed by NOT, AND, and then OR.



BOOLEAN FUNCTION

- A Boolean function can be represented in a truth table.

$$f(x,y,z) = (x + y')z + x'$$



$$f(0,0,0) = (0 + 1)0 + 1 = 1$$

$$f(0,0,1) = (0 + 1)1 + 1 = 1$$

$$f(0,1,0) = (0 + 0)0 + 1 = 1$$

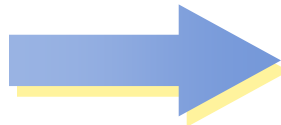
$$f(0,1,1) = (0 + 0)1 + 1 = 1$$

$$f(1,0,0) = (1 + 1)0 + 0 = 0$$

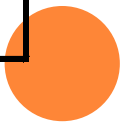
$$f(1,0,1) = (1 + 1)1 + 0 = 1$$

$$f(1,1,0) = (1 + 0)0 + 0 = 0$$

$$f(1,1,1) = (1 + 0)1 + 0 = 1$$



x	y	z	$f(x,y,z)$
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1



GATE IMPLEMENTATION OF A FUNCTION

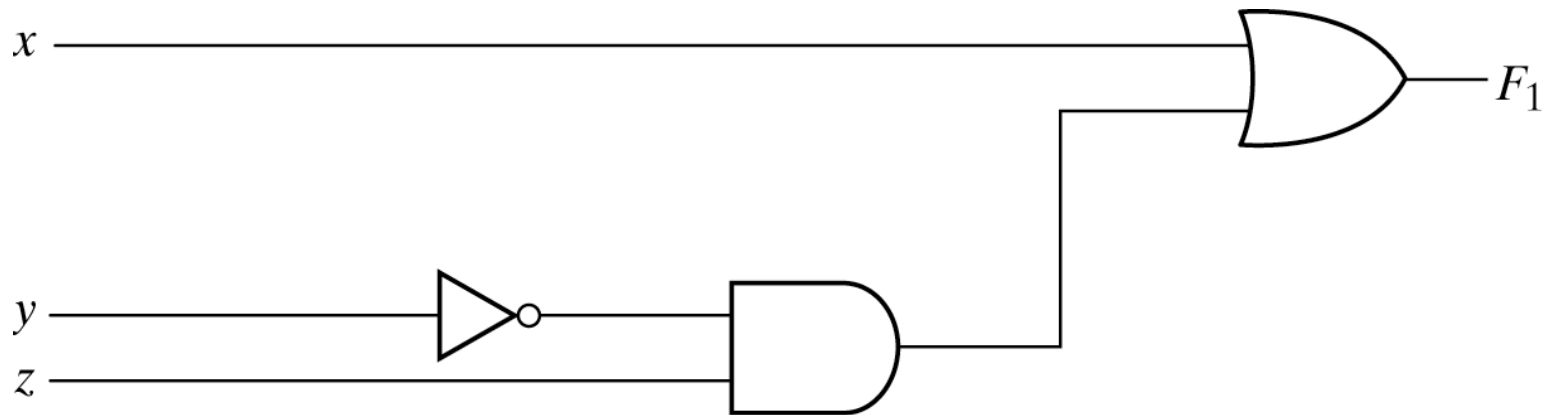
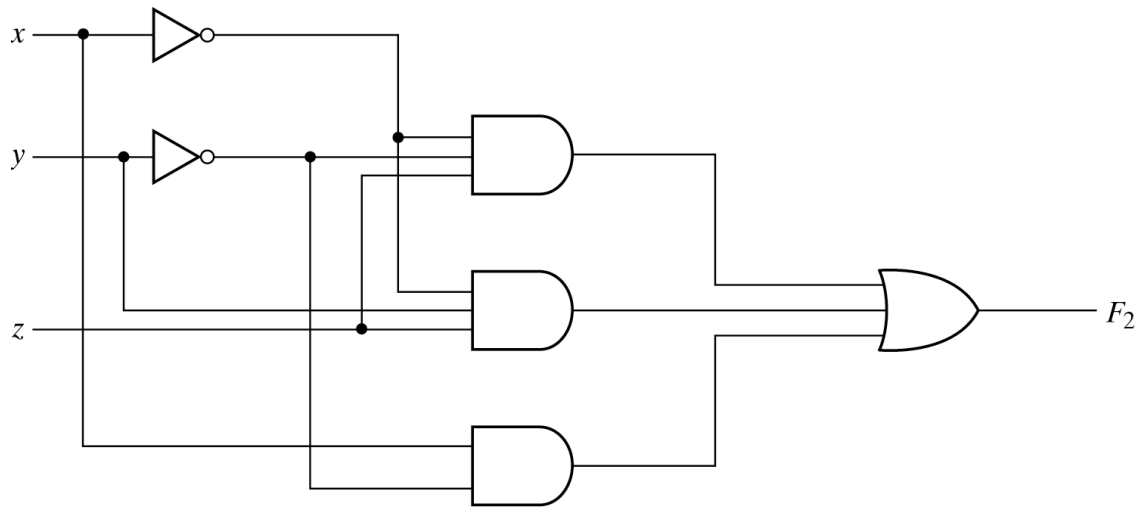


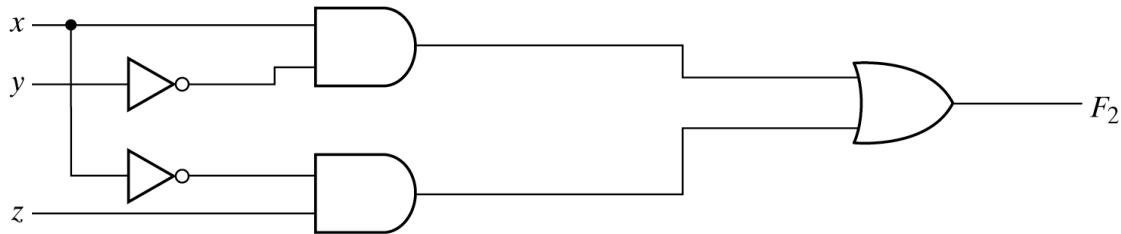
Fig. 2-1 Gate implementation of $F_1 = x + y'z$



GATE IMPLEMENTATION OF A FUNCTION



(a) $F_2 = x'y'z + x'yz + xy'$



(b) $F_2 = xy' + x'z$

Fig. 2-2 Implementation of Boolean function F_2 with gates



BOOLEAN ALGEBRA

- Algebra for binary values
 - **Boolean Algebra:** Developed by George Boole in 1854
- Why study Boolean Algebra?
 - To find the simplest circuit implementation with the smallest number of gates or wires
 - Simpler circuits are cheaper to build, consume less power, and run faster than complex circuits



ALGEBRA

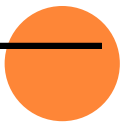
- What is an algebra?
 - Mathematical system consisting of
 - Set of elements
 - Set of operators
 - Axioms or postulates: facts that can be taken as true; they do not require proof



AXIOMATIC DEFINITION OF BOOLEAN ALGEBRA

- A Boolean algebra requires
 - A set of elements ***B***, consisting of two elements (0 and 1)
 - Two binary operations OR and AND
 - The axioms below must always be true

1. $x + y \in B$	$x \bullet y \in B$	Closure
2. $x + 0 = x$	$x \bullet 1 = x$	Identity
3. $x + y = y + x$	$x \bullet y = y \bullet x$	Commutativity
4. $x(y + z) = xy + xz$	$x + yz = (x + y)(x + z)$	Distributivity
5. $x + x' = 1$	$x \bullet x' = 0$	Complement
6. At least 2 elements: $x, y \in B$ such that $x \neq y$		Cardinality



AXIOMATIC DEFINITION OF BOOLEAN ALGEBRA

- Based on axiom #5, we can develop a unary (one-argument) operation NOT

x	x'
0	1
1	0



AXIOMATIC DEFINITION OF BOOLEAN ALGEBRA

- The distributive laws

x	y	z	$y+z$	$x \cdot (y+z)$	$x \cdot y$	$x \cdot z$	$(x \cdot y) + (x \cdot z)$
0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	0	0	0	0
1	0	0	0	0	0	0	0
1	0	1	1	1	0	1	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1



DUALITY PRINCIPLE

- If an expression is valid in Boolean algebra, the dual of that expression is also valid
- To form the dual of an expression, replace
 - all + operators with . operators,
 - all . operators with + operators,
 - all ones with zeros, and
 - all zeros with ones
- Form the dual of the expression
$$a + (bc) = (a + b)(a + c)$$
- Following the replacement rules...
$$a(b + c) = ab + ac$$



DUALITY PRINCIPLE

- The left and right columns of axioms are **duals**
 - exchange all ANDs with ORs, and 0s with 1s

1. $x + y \in B$	$x \bullet y \in B$	Closure
2. $x + 0 = x$	$x \bullet 1 = x$	Identity
3. $x + y = y + x$	$x \bullet y = y \bullet x$	Commutativity
4. $x(y + z) = xy + xz$	$x + yz = (x + y)(x + z)$	Distributivity
5. $x + x' = 1$	$x \bullet x' = 0$	Complement
6. At least 2 elements: $x, y \in B$ such that $x \neq y$		Cardinality



BASIC THEOREMS OF BOOLEAN ALGEBRA

- In addition to the axioms, additional laws can be derived; they are called theorems of Boolean Algebra
- These theorems are useful in performing algebraic manipulations of Boolean expressions

1. $x + x = x$	$x \bullet x = x$	Idempotency
2. $x + 1 = 1$	$x \bullet 0 = 0$	
3. $yx + x = x$	$(y + x) \bullet x = x$	Absorption
4. $(x')' = x$		Involution
5. $x + (y + z) = (x + y) + z$	$x(yz) = (xy)z$	Associative
6. $(x + y)' = x'y'$	$(xy)' = x' + y'$	DeMorgan's



PROOF OF $x+x=x$

- We can only use Huntington postulates:

Post. 2:	(a) $x+0=x$, (b) $x \cdot 1=x$
Post. 3:	(a) $x+y=y+x$, (b) $x \cdot y=y \cdot x$
Post. 4:	(a) $x(y+z) = xy+xz$, (b) $x+yz = (x+y)(x+z)$
Post. 5:	(a) $x+x'=1$, (b) $x \cdot x'=0$

- Show that $x+x=x$.

$$\begin{aligned}x+x &= (x+x) \cdot 1 && \text{by 2(b)} \\&= (x+x)(x+x') && \text{by 5(a)} \\&= x+xx' && \text{by 4(b)} \\&= x+0 && \text{by 5(b)} \\&= x && \text{by 2(a)}\end{aligned}$$

- We can now use Theorem 1(a) in future proofs



PROOF OF $x \cdot x = x$

- Similar to previous proof

Post. 2: (a) $x+0=x$, (b) $x \cdot 1=x$

Post. 3: (a) $x+y=y+x$, (b) $x \cdot y=y \cdot x$

Post. 4: (a) $x(y+z) = xy+xz$,
(b) $x+yz = (x+y)(x+z)$

Post. 5: (a) $x+x'=1$, (b) $x \cdot x'=0$

Th. 1: (a) $x+x=x$

- Show that $x \cdot x = x$.

$$x \cdot x = xx+0 \quad \text{by 2(a)}$$

$$= xx+xx' \quad \text{by 5(b)}$$

$$= x(x+x') \quad \text{by 4(a)}$$

$$= x \cdot 1 \quad \text{by 5(a)}$$

$$= x \quad \text{by 2(b)}$$



PROOF OF $x+1=1$

- Theorem 2(a): $x + 1 = 1$

$$x + 1 = 1 \cdot (x + 1) \quad \text{by 2(b)}$$

$$= (x + x')(x + 1) \quad 5(a)$$

$$= x + x' 1 \quad 4(b)$$

$$= x + x' \quad 2(b)$$

$$= 1$$

Post. 2: (a) $x+0=x$, (b) $x \cdot 1=x$

Post. 3: (a) $x+y=y+x$, (b) $x \cdot y=y \cdot x$

Post. 4: (a) $x(y+z) = xy+xz$,
(b) $x+yz = (x+y)(x+z)$

Post. 5: (a) $x+x'=1$, (b) $x \cdot x'=0$

Th. 1: (a) $x+x=x$, (b) $x \cdot x = x$

- Theorem 2(b): $x \cdot 0 = 0$ by duality

- Theorem 4: $(x')' = x$

- Postulate 5 defines the complement of x , $x + x' = 1$ and $x x' = 0$
- The complement of x' is x is also $(x')'$



ABSORPTION PROPERTY (COVERING)

Theorem 6(a): $x + xy = x$

$$x + xy = x \cdot 1 + xy \text{ by } 2(b)$$

$$= x(1 + y) \quad 4(a)$$

$$= x(y + 1) \quad 3(a)$$

$$= x \cdot 1 \quad \text{Th } 2(a)$$

$$= x \quad 2(b)$$

Post. 2: (a) $x+0=x$, (b) $x \cdot 1=x$

Post. 3: (a) $x+y=y+x$, (b) $x \cdot y=y \cdot x$

Post. 4: (a) $x(y+z) = xy+xz$,
(b) $x+yz = (x+y)(x+z)$

Post. 5: (a) $x+x'=1$, (b) $x \cdot x'=0$

Th. 1: (a) $x+x=x$, (b) $x \cdot x =x$

Th. 2: (a) $x + 1 = 1$, (b) $x \cdot 0 =0$

- Theorem 6(b): $x(x + y) = x$ by duality
- By means of truth table (another way to proof)



ABSORPTION PROPERTY (COVERING)

Theorem 6(a): $x + xy = x$

x	y	xy	$x+xy$
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1



DEMORGAN'S THEOREM

- Theorem 5(a): $(x + y)' = x'y'$
- Theorem 5(b): $(xy)' = x' + y'$
- By means of truth table

x	y	x'	y'	$x+y$	$(x+y)'$	$x'y'$	xy	$x'+y'$	$(xy)'$
0	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	0	0	1	1
1	0	0	1	1	0	0	0	1	1
1	1	0	0	1	0	0	1	0	0



CONSENSUS THEOREM

1. $xy + x'z + yz = xy + x'z$
2. $(x+y) \cdot (x'+z) \cdot (y+z) = (x+y) \cdot (x'+z) \quad \text{-- (dual)}$

○ **Proof:**

$$\begin{aligned} xy + x'z + yz &= xy + x'z + (x+x')yz \\ &= xy + x'z + xyz + x'yz \\ &= (xy + xyz) + (x'z + x'zy) \\ &= xy(1+z) + x'z(1+y) \\ &= xy + x'z \end{aligned}$$

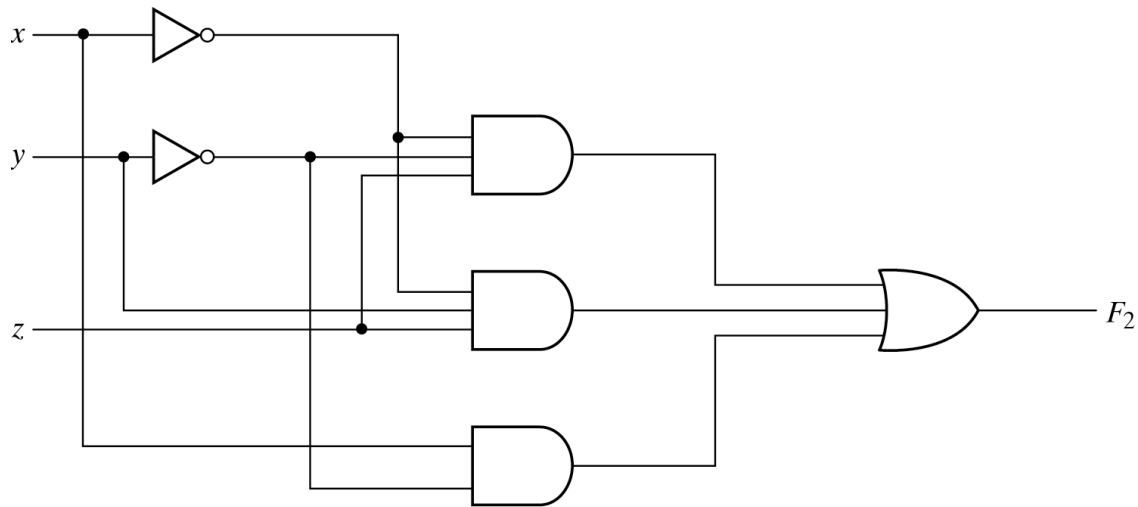


GATE IMPLEMENTATION OF A FUNCTION

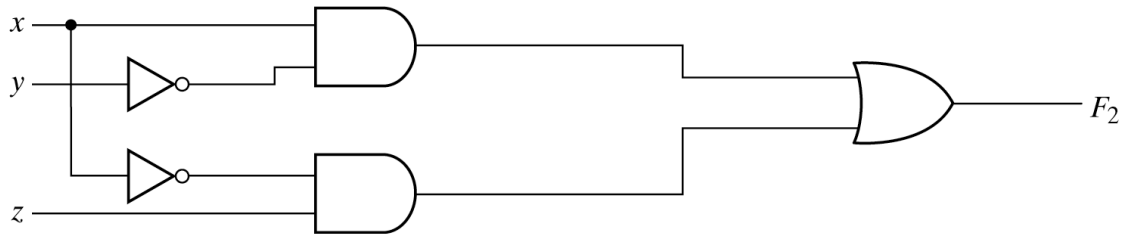
$$\begin{aligned} F_2 &= x'y'z + x'yz + xy' = x'z(y'+y) + xy' \\ &= x'z + xy' \end{aligned}$$



GATE IMPLEMENTATION OF A FUNCTION



(a) $F_2 = x'y'z + x'yz + xy'$



(b) $F_2 = xy' + x'z$

Fig. 2-2 Implementation of Boolean function F_2 with gates



PRACTICE

- Simplify the following Boolean expressions to a minimum number of literals:
 - $xyz + x'y + xyz'$
 - $(A + B)'(A' + B')'$
- $xyz + x'y + xyz' = xy(z + z') + x'y = xy + x'y = y$
- $(A + B)'(A' + B')' = (A'B')(A B) = 0$



COMPLEMENT OF A FUNCTION

- Applying DeMorgan's theorems:

$$f(x,y,z) = x(y'z' + yz)$$

$$\begin{aligned} f'(x,y,z) &= (x(y'z' + yz))' \text{ [complement both sides]} \\ &= x' + (y'z' + yz)' \text{ [because } (xy)' = x' + y' \text{]} \\ &= x' + (y'z')'(yz)' \text{ [because } (x + y)' = x'y' \text{]} \\ &= x' + (y + z)(y' + z') \text{ [because } (xy)' = x' + y', \text{ twice} \text{]} \end{aligned}$$

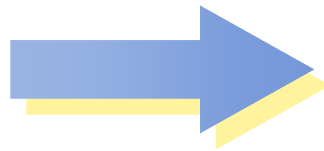


COMPLEMENT OF A FUNCTION

- The complement of a function always outputs 0 where the original function outputted 1, and 1 where the original produced 0

$$f(x,y,z) = (x + y')z + x'$$

x	y	z	$f(x,y,z)$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



x	y	z	$f'(x,y,z)$
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0



COMPLEMENT OF A FUNCTION

- By taking the dual of f and complementing each literal:
 - If $f(x,y,z) = x(y'z' + yz)...$
 - ...the dual of f is $x + (y' + z')(y + z)...$
 - ...then complementing each literal gives
 $x' + (y + z)(y' + z')...$
 - ...so $f'(x,y,z) = x' + (y + z)(y' + z')$



PRACTICE

- Find the complement of the following expressions:

- $xy' + x'y$

- $[(x' + y + z')(x + y')(x + z)]$

- $F' = (xy' + x'y)'$

$$= (xy')'(x'y)'$$

$$= (x' + y)(x + y')$$

$$= xy + x'y'$$

- $F' = [(x' + y + z')(x + y')(x + z)]'$

$$= (x' + y + z')' + (x + y')' + (x + z)'$$

$$= xy'z + x'y + x'z'$$



GENERALIZED DEMORGAN'S THEOREM

- $(A + B + C)'$ $= (A + x)'$ let $B + C = x$
 $= A'x'$
 $= A'(B + C)'$
 $= A'(B'C')$
 $= A'B'C'$

- Complement of a function is obtained
 - Interchanging AND and OR operators
 - Complementing each literal



MINTERMS

- A minterm is an AND term in which every variable or its complement occurs once
 - $F(x,y)$ has 4 minterms $x'y'$, $x'y$, xy' , xy
- An n variable function has 2^n valid minterms
- A minterm equals 1 at exactly one input combination and is equal to 0 otherwise
 - Example: $x'y'z' = 1$ only when $x=0$, $y=0$, $z=0$



MAXTERMS

- A maxterm is an OR term in which every variable or its complement occurs once
 - $F(x,y)$ has 4 maxterms $x'+y'$, $x'+y$, $x+y'$, $x+y$
- An n variable function has 2^n valid maxterms
- A maxterm equals 0 at exactly one input combination and is equal to 1 otherwise
 - Example: $(x+y+z) = 0$ only when $x=0$, $y=0$, $z=0$



EXAMPLE: THREE BINARY VARIABLES

Table 2-3:

Minterms and Maxterms for Three Binary Variables

x	y	z	minterms		Maxterms	
0	0	0	$x'y'z'$	m_0	$x+y+z$	M_0
0	0	1	$x'y'z$	m_1	$x+y+z'$	M_1
0	1	0	$x'yz'$	m_2	$x+y'+z$	M_2
0	1	1	$x'yz$	m_3	$x+y'+z'$	M_3
1	0	0	$xy'z'$	m_4	$x'+y+z$	M_4
1	0	1	$xy'z$	m_5	$x'+y+z'$	M_5
1	1	0	xyz'	m_6	$x'+y'+z$	M_6
1	1	1	xyz	m_7	$x'+y'+z'$	M_7



CANONICAL FORM

- Any boolean function that is expressed as a **sum of minterms** or as a **product of maxterms** is said to be in its canonical form.



SUM OF MINTERMS

x	y	z	F_1	F_2
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1
0	1	1	0	1
1	0	0	1	0
1	0	1	1	0
1	1	0	1	0
1	1	1	1	0

- $F_1(x, y, z) = \sum(1,4,5,6,7) = m_1 + m_4 + m_5 + m_6 + m_7$
 $= x' y' z + x y' z' + x y' z + x y z' + x y z$



SUM OF MINTERMS: EXAMPLE

- $F = x + yz$
$$\begin{aligned} &= x(y + y')(z + z') + (x + x')yz \\ &= xyz + xyz' + xy'z + xy'z' + xyz + x'yz \\ &= x'yz + xy'z' + xy'z + xyz' + xyz \\ &= \Sigma(3,4,5,6,7) \end{aligned}$$
- Or convert the expression into truth-table and then read the minterms from the table



PRODUCT OF MAXTERMS

- A Boolean function can be expressed algebraically from a given truth table
 - by forming a maxterm for each combination of the variables that produces a 0 in the function, and
 - then taking the AND of all those maxterms.



PRODUCT OF MAXTERMS

x	y	z	F_1	F_2
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1
0	1	1	0	1
1	0	0	1	0
1	0	1	1	0
1	1	0	1	0
1	1	1	1	0

- $F_1(x, y, z) = \prod(0,2,3) = M_0 \times M_2 \times M_3$
 $= (x + y + z)(x + y' + z)(x + y' + z')$



PRODUCT OF MAXTERMS: EXAMPLE

- $F = xy + x'z = (xy+x')(xy+z)$
 $= (x+x')(y+x')(x+z)(y+z) = (x'+y)(x+z)(y+z)$

$$x'+y = x'+y+zz' = (x'+y+z)(x'+y+z')$$

$$x+z = x+z+yy' = (x+y+z)(x+y'+z)$$

$$y+z = y+z+xx' = (x+y+z)(x'+y+z)$$

$$F = (x+y+z)(x+y'+z)(x'+y+z)(x'+y+z')$$
$$= M_0 M_2 M_4 M_5 = \prod(0, 2, 4, 5)$$

- Or convert the expression into truth-table and then read the maxterms from the table



CONVERSION BETWEEN CANONICAL FORMS

- Conversion between minterms and maxterms

$$m_0 = x'y'z' = (x+y+z)' = (M_0)'$$

- In general, $m_i = (M_i)'$

- Sum of minterms $\rightarrow$ Product of maxterms:

$$f = \Sigma(0,1,2,3,6)$$

$$f' = \Sigma(4,5,7) = m_4 + m_5 + m_7$$

$$(f')' = (m_4 + m_5 + m_7)'$$

$$f = m_4' m_5' m_7' \text{ [DeMorgan's law]}$$

$$= M_4 M_5 M_7 = \Pi(4,5,7)$$



CONVERSION BETWEEN CANONICAL FORMS

- In general, to convert from one canonical form to another, interchange the symbols Σ and $\prod$ and list those numbers missing from the original form.
 - *Example:* $f = \Sigma(0,1,2,3,6) = \prod(4,5,7)$



PRACTICE

- Express the following function (four variables: A , B , C , D) as a sum of minterms and as a product of maxterms:
 - $F = B'D + A'D + BD$
- $F = \sum(1, 3, 5, 7, 9, 11, 13, 15)$
 $= \prod(0, 2, 4, 6, 8, 10, 12, 14)$



STANDARD FORM

- Any boolean function that is expressed as a **sum of products (SOP)** or as a **product of sums (POS)**, where each product-term or sum-term may contain one, two, or any number of variables, is said to be in its **standard form**.
- SOP: $f(x,y,z) = xy + x'yz + xy'z$
- POS: $f(x,y,z) = (x' + y')(x + y' + z')(x' + y + z')$



NON-STANDARD AND STANDARD FORMS

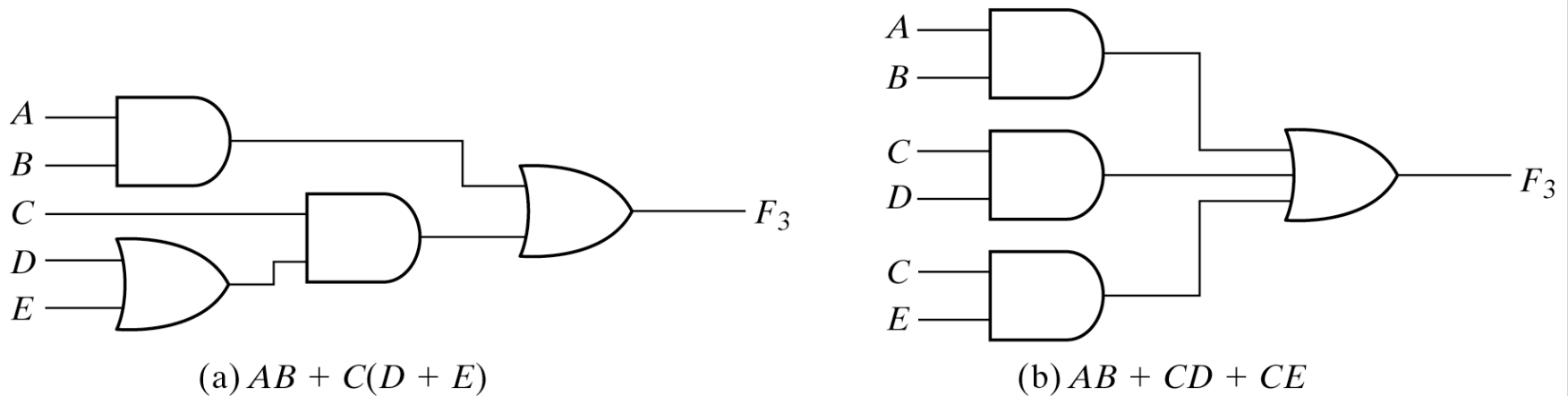


Fig. 2-4 Three- and Two-Level implementation



PRACTICE

- Convert the following expression into sum of products and product of sums:

- $(AB + C)(B + C'D)$

- $(AB + C)(B + C'D)$
 $= AB + BC + ABC'D + CC'D$
 $= AB(1 + C'D) + BC$
 $= AB + BC$ (*SOP form*)
 $= B(A + C)$ (*POS form*)



CANONICAL AND STANDARD FORMS

- Canonical forms
 - Sum of minterms (SOM)
 - Product of maxterms (POM)
- Standard forms (may use less gates)
 - Sum of products (SOP)
 - Product of sums (POS)
- $F = ab + a'$ (already **sum of products:SOP**)
- $F = ab + a'(b + b')$ (expanding term)
- $F = ab + a'b + a'b'$ (it is **canonical form:SOM**)



OTHER LOGIC OPERATIONS

- 2^n rows in the truth table of n binary variables.
- 2^{2^n} functions for n binary variables.
- 16 functions of two binary variables.

Table 2.7

Truth Tables for the 16 Functions of Two Binary Variables

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1



BOOLEAN EXPRESSIONS

Table 2.8

Boolean Expressions for the 16 Functions of Two Variables

Boolean Functions	Operator Symbol	Name	Comments
$F_0 = 0$		Null	Binary constant 0
$F_1 = xy$	$x \cdot y$	AND	x and y
$F_2 = xy'$	x/y	Inhibition	x , but not y
$F_3 = x$		Transfer	x
$F_4 = x'y$	y/x	Inhibition	y , but not x
$F_5 = y$		Transfer	y
$F_6 = xy' + x'y$	$x \oplus y$	Exclusive-OR	x or y , but not both
$F_7 = x + y$	$x + y$	OR	x or y
$F_8 = (x + y)'$	$x \downarrow y$	NOR	Not-OR
$F_9 = xy + x'y'$	$(x \oplus y)'$	Equivalence	x equals y
$F_{10} = y'$	y'	Complement	Not y
$F_{11} = x + y'$	$x \subset y$	Implication	If y , then x
$F_{12} = x'$	x'	Complement	Not x
$F_{13} = x' + y$	$x \supset y$	Implication	If x , then y
$F_{14} = (xy)'$	$x \uparrow y$	NAND	Not-AND
$F_{15} = 1$		Identity	Binary constant 1



DIGITAL LOGIC GATES

- Boolean expression: AND, OR and NOT operations
- Constructing gates of other logic operations
 - The feasibility and economy;
 - The possibility of extending gate's inputs;
 - The basic properties of the binary operations (commutative and associative);
 - The ability of the gate to implement Boolean functions.



SUMMARY OF LOGIC GATES

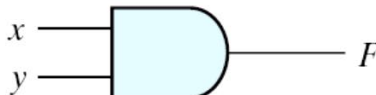
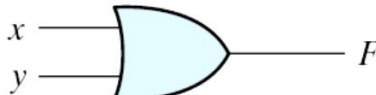
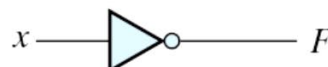
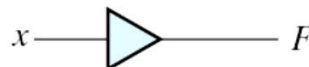
Name	Graphic symbol	Algebraic function	Truth table															
AND		$F = xy$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F = x + y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	1
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
Inverter		$F = x'$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	x	F	0	1	1	0									
x	F																	
0	1																	
1	0																	
Buffer		$F = x$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	x	F	0	0	1	1									
x	F																	
0	0																	
1	1																	

Figure 2.5 Digital logic gates



SUMMARY OF LOGIC GATES

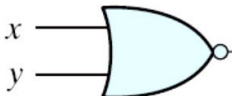
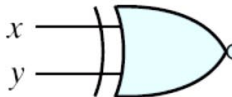
NAND	 $F = (xy)'$	<table> <tr> <th>x</th><th>y</th><th>F</th></tr> <tr><td>0</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </table>	x	y	F	0	0	1	0	1	1	1	0	1	1	1	0
x	y	F															
0	0	1															
0	1	1															
1	0	1															
1	1	0															
NOR	 $F = (x + y)'$	<table> <tr> <th>x</th><th>y</th><th>F</th></tr> <tr><td>0</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	0
x	y	F															
0	0	1															
0	1	0															
1	0	0															
1	1	0															
Exclusive-OR (XOR)	 $F = xy' + x'y$ $= x \oplus y$	<table> <tr> <th>x</th><th>y</th><th>F</th></tr> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	0
x	y	F															
0	0	0															
0	1	1															
1	0	1															
1	1	0															
Exclusive-NOR or equivalence	 $F = xy + x'y'$ $= (x \oplus y)'$	<table> <tr> <th>x</th><th>y</th><th>F</th></tr> <tr><td>0</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>1</td><td>1</td></tr> </table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	1
x	y	F															
0	0	1															
0	1	0															
1	0	0															
1	1	1															

Figure 2.5 Digital logic gates



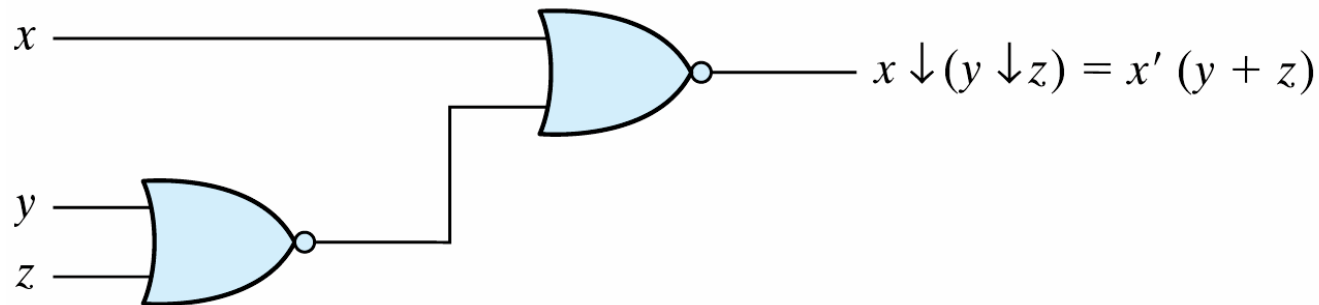
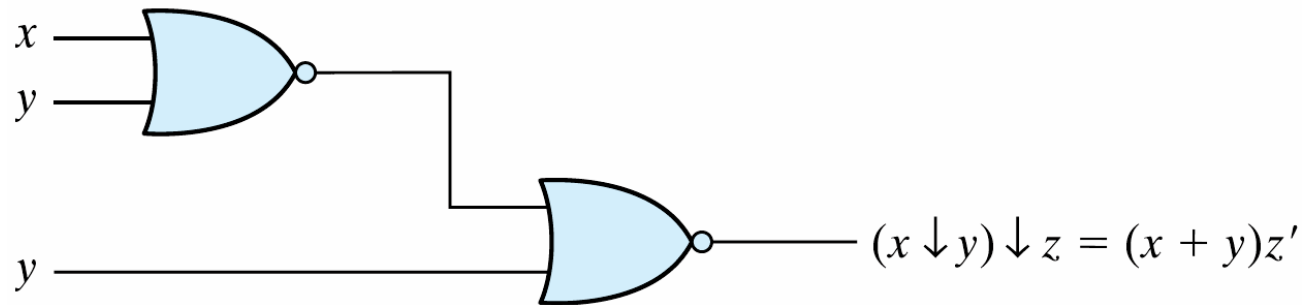
MULTIPLE INPUTS

- Extension to multiple inputs
 - A gate can be extended to multiple inputs.
 - If its binary operation is commutative and associative.
 - AND and OR are commutative and associative.
 - OR
 - $x+y = y+x$
 - $(x+y)+z = x+(y+z) = x+y+z$
 - AND
 - $xy = yx$
 - $(x\ y)z = x(y\ z) = x\ y\ z$



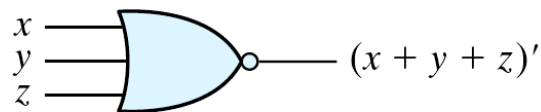
MULTIPLE INPUTS

- NAND and NOR are commutative but not associative $\rightarrow$ they are not extendable.

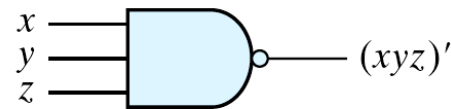


MULTIPLE INPUTS

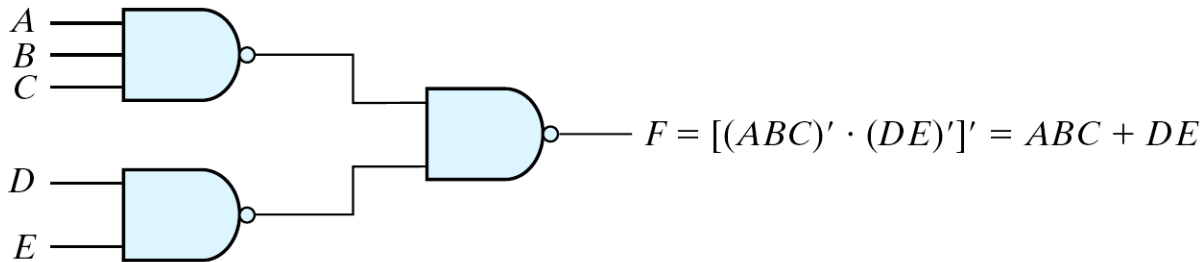
- Multiple NOR = a complement of OR gate,
Multiple NAND = a complement of AND.
- The cascaded NAND operations = sum of products.
- The cascaded NOR operations = product of sums.



(a) 3-input NOR gate



(b) 3-input NAND gate

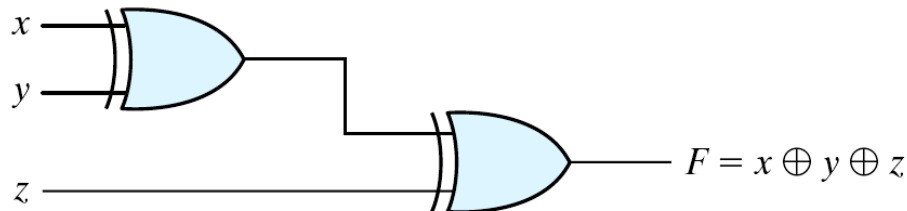


(c) Cascaded NAND gates

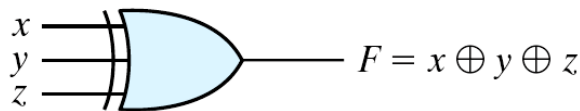


MULTIPLE INPUTS

- The XOR and XNOR gates are commutative and associative.
- Multiple-input XOR gates are uncommon
- XOR is an odd function: it is equal to 1 if the inputs variables have an odd number of 1's.



(a) Using 2-input gates



(b) 3-input gate

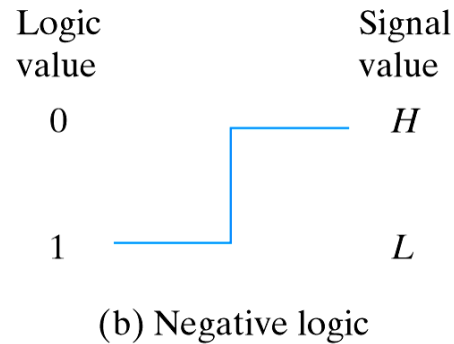
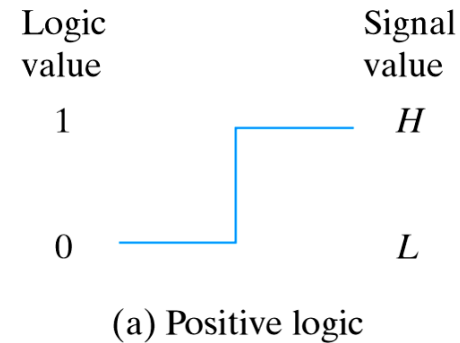
x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(c) Truth table



POSITIVE AND NEGATIVE LOGIC

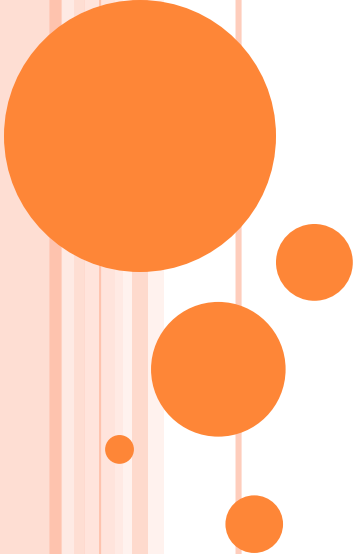
- Positive and Negative Logic
 - Two signal values $\Leftrightarrow$ two logic values
 - Positive logic: $H=1$; $L=0$
 - Negative logic: $H=0$; $L=1$



SYLLABUS

- Chapter 2 (Excluding Section 2.9)





CSE 205: DIGITAL LOGIC DESIGN

Dr. Tanzima Hashem

Professor

CSE, BUET

NUMBER SYSTEMS

- Consists of **TWO Things**:
 - A **BASE** or **RADIX** Value
 - A **SET** of **DIGITS**
 - *Digits are symbols representing all values **less than the radix value**.*
- Example is the Common Decimal System:
 - $\text{RADIX (BASE)} = 10$
 - Digit Set = $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$



DECIMAL NUMBER SYSTEMS

- Consider: 5032.21

$$\begin{aligned}(5032.21)_{10} &= 5 \times (10)^3 + 0 \times (10)^2 + 3 \times (10)^1 + 2 \times (10)^0 + 2 \times (10)^{-1} + 1 \times (10)^{-2} \\ &= 5000 + 0 + 30 + 2 + 0.2 + 0.01\end{aligned}$$

- Other Notation: $(5032.21)_{10}$
- In general, a number expressed in a base- r system has coefficients multiplied by powers of r :

$$a_n \cdot r^n + a_{n-1} \cdot r^{n-1} + \dots + a_1 \cdot r^1 + a_0 + a_{-1} \cdot r^{-1} + a_{-2} \cdot r^{-2} + \dots + a_{-m} \cdot r^{-m}$$



OTHER NUMBER SYSTEMS

- Binary

- Radix = $(2)_{10}$
- Digit Set = $\{0,1\}$

- Octal

- Radix = $(8)_{10}$
- Digit Set = $\{0,1,2,3,4,5,6,7\}$

- Hexadecimal

- Radix = $(16)_{10}$
- Digit Set = $\{0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F\}$



BINARY NUMBER SYSTEMS

- Binary
 - Radix = $(2)_{10}$
 - Digit Set = $\{0,1\}$

- Binary to Decimal:

$$\begin{aligned}(1101.01)_2 &= 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} \\ &= (13.25)_{10}\end{aligned}$$



BINARY TO DECIMAL: PRACTICE

a) $0110_2 = ?$ 6_{10}

b) $11010_2 = ?$ 26_{10}

c) $0110101_2 = ?$ 53_{10}

d) $11010011_2 = ?$ 211_{10}



OCTAL NUMBER SYSTEMS

- Octal

- Radix = $(8)_{10}$
- Digit Set = $\{0,1,2,3,4,5,6,7\}$

- Octal to Decimal:

$$\begin{aligned}(15.2)_8 &= 1 \times 8^1 + 5 \times 8^0 + 2 \times 8^{-1} \\ &= (13.25)_{10}\end{aligned}$$



HEXADECIMAL NUMBER SYSTEMS

- Hexadecimal
 - Radix = $(16)_{10}$
 - Digit Set = $\{0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F\}$
- Hexadecimal to Decimal:

$$\begin{aligned}(D.4)_{16} &= D \times (16)^0 + 4 \times (16)^{-1} \\ &= (13.25)_{10}\end{aligned}$$



DECIMAL (*INTEGER*) TO BINARY

- Divide the number by the 'Base' (=2)
- Take the remainder (either 0 or 1) as a coefficient
- Take the quotient and repeat the division

Example: $(13)_{10}$

	Quotient	Remainder	Coefficient
13 / 2 =	6	1	$a_0 = 1$
6 / 2 =	3	0	$a_1 = 0$
3 / 2 =	1	1	$a_2 = 1$
1 / 2 =	0	1	$a_3 = 1$

Answer: $(13)_{10} = (a_3 a_2 a_1 a_0)_2 = (1101)_2$

MSB LSB



DECIMAL TO BINARY: PRACTICE

a) $13_{10} = ?$ $1\ 1\ 0\ 1_2$

b) $22_{10} = ?$ $1\ 0\ 1\ 1\ 0_2$

c) $43_{10} = ?$ $1\ 0\ 1\ 0\ 1\ 1_2$

d) $158_{10} = ?$ $1\ 0\ 0\ 1\ 1\ 1\ 1\ 0_2$



DECIMAL (*FRACTION*) TO BINARY

- Multiply the number by the 'Base' (=2)
- Take the integer (either 0 or 1) as a coefficient
- Take the resultant fraction and repeat the division

Example: $(0.625)_{10}$

		Integer	Fraction	Coefficient
0.625	$* 2 =$	1	$. 25$	$a_{-1} = 1$
0.25	$* 2 =$	0	$. 5$	$a_{-2} = 0$
0.5	$* 2 =$	1	$. 0$	$a_{-3} = 1$

Answer: $(0.625)_{10} = (0.a_{-1}a_{-2}a_{-3})_2 = (0.101)_2$

$\uparrow$ $\uparrow$
MSB LSB



DECIMAL TO OCTAL CONVERSION

Example: $(175)_{10}$

	Quotient	Remainder	Coefficient
$175 / 8 =$	21	7	$a_0 = 7$
$21 / 8 =$	2	5	$a_1 = 5$
$2 / 8 =$	0	2	$a_2 = 2$

Answer: $(175)_{10} = (a_2 a_1 a_0)_8 = (257)_8$

Example: $(0.3125)_{10}$

	Integer	Fraction	Coefficient
$0.3125 * 8 =$	2	. 5	$a_{-1} = 2$
$0.5 * 8 =$	4	. 0	$a_{-2} = 4$

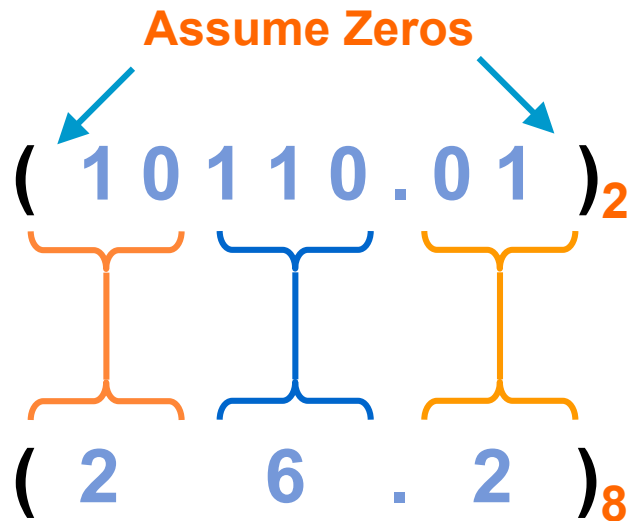
Answer: $(0.3125)_{10} = (0.a_{-1} a_{-2} a_{-3})_8 = (0.24)_8$



BINARY – OCTAL CONVERSION

- $8 = 2^3$
- Each group of 3 bits represents an octal digit

Example:



Octal	Binary
0	0 0 0
1	0 0 1
2	0 1 0
3	0 1 1
4	1 0 0
5	1 0 1
6	1 1 0
7	1 1 1

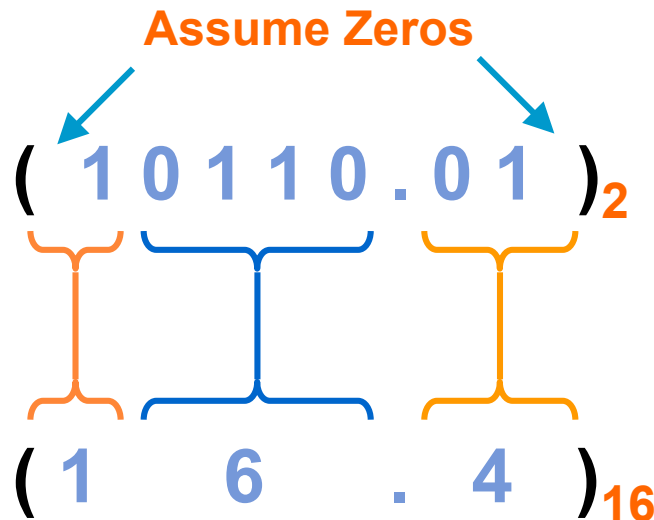
Works **both** ways (Binary to Octal & Octal to Binary)



BINARY – HEXADECIMAL CONVERSION

- $16 = 2^4$
- Each group of 4 bits represents a hexadecimal digit

Example:



Hex	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

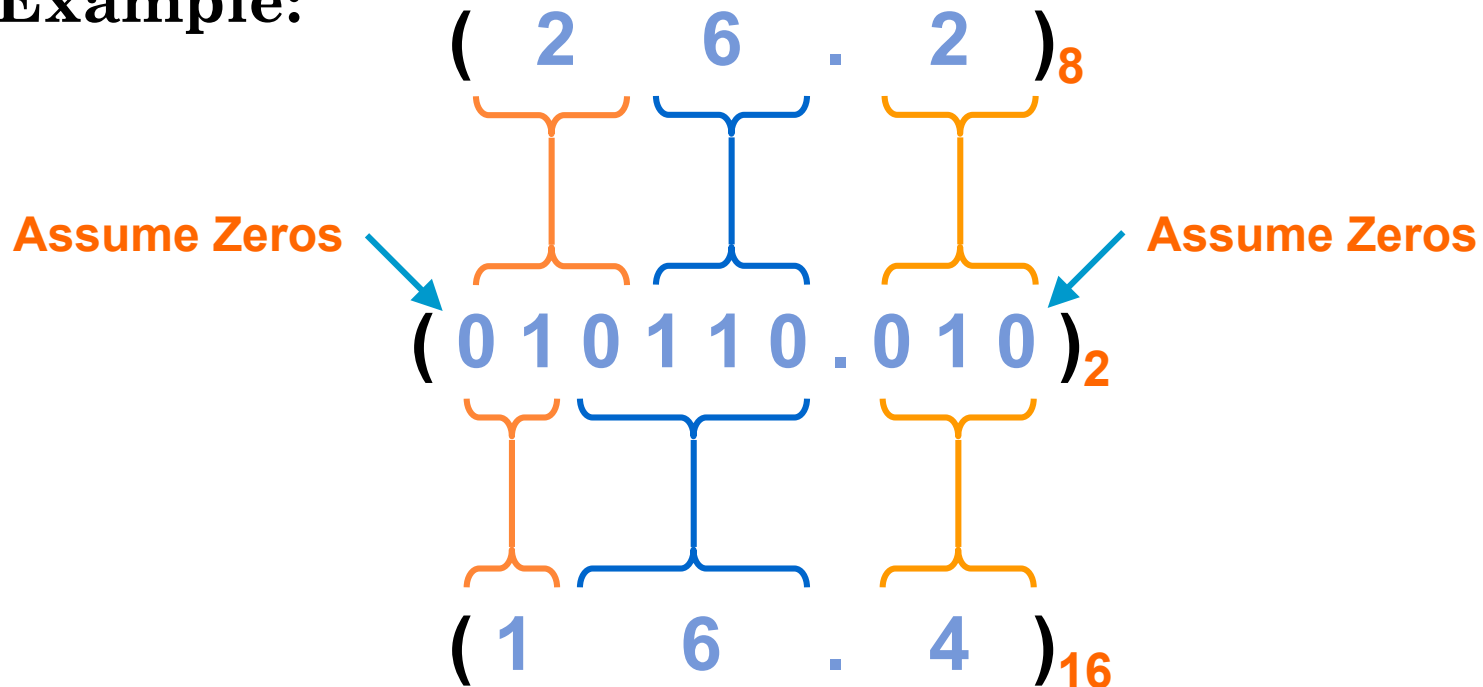
Works **both** ways (Binary to Hex & Hex to Binary)



OCTAL – HEXADECIMAL CONVERSION

- Convert to **Binary** as an intermediate step

Example:



Works **both** ways (Octal to Hex & Hex to Octal) ●

BINARY ADDITION

	1	1	1	1	1	1		
		1	1	1	1	0	1	= 61
+			1	0	1	1	1	= 23
	1	0	1	0	1	0	0	= 84

 $\geq (2)_{10}$



BINARY SUBTRACTION

			1		2			= (10) ₂
	0	2	2	0	0	2		
	1	0	0	1	1	0	1	= 77
-			1	0	1	1	1	= 23
	0	1	1	0	1	1	0	= 54



COMPLEMENTS

- They are used to simplify the subtraction operation
- Two types (for each *base-r* system)
 - Diminishing radix complement ($r-1$'s complement)
 - Radix complement (r 's complement)

For n -digit number N

$$(r^n - 1) - N \longrightarrow r-1\text{'s complement}$$

$$r^n - N \longrightarrow r\text{'s complement}$$



DIMINISHED RADIX COMPLEMENT

○ Example for 6-digit decimal numbers:

- 9's complement is $(r^n - 1) - N = (10^6 - 1) - N = 999999 - N$
- 9's complement of 546700:

$$999999 - 546700 = 453299$$

○ Example for 7-digit binary numbers:

- 1's complement is $(r^n - 1) - N = (2^7 - 1) - N = 1111111 - N$
- 1's complement of 1011000:

$$1111111 - 1011000 = 0100111$$



RADIX COMPLEMENT

- The r 's complement of an n -digit number N in base r is defined as $r^n - N$ for $N \neq 0$ and as 0 for $N = 0$.

- The r 's complement can also be obtained by adding 1 to the $(r - 1)$'s complement, since

$$r^n - N = [(r^n - 1) - N] + 1.$$

- Decimal Number: $10^n - N$

- Example: 10's complement of 246700 is 753300



RADIX COMPLEMENT (BINARY NUMBER)

- Take 1's complement then add 1

OR

- Toggle all bits to the left of the first '1' from the right

- **Example:**

$$\begin{array}{r} 1\ 0\ 1\ 1\ 0\ 0\ 0\ 0 \\ 0\ 1\ 0\ 0\ 1\ 1\ 1\ 1 \\ + 1 \\ \hline 0\ 1\ 0\ 1\ 0\ 0\ 0\ 0 \end{array}$$

$$1\ 0\ 1\ 1\ 0\ 0\ 0\ 0$$

$$0\ 1\ 0\ 1\ 0\ 0\ 0\ 0$$



SUBTRACTION WITH COMPLEMENTS

- $M - N$
 - Add M to r 's complement of N
 - $Sum = M + (r^n - N) = M - N + r^n$
 - If $M > N$, Sum will have an end carry r^n , discard it
 - If $M < N$, Sum will not have an end carry and
 - $Sum = r^n - (N - M)$ (r 's complement of $N - M$)
 - So $M - N = - (r\text{'s complement of } Sum)$



SUBTRACTION WITH COMPLEMENTS

- 65438 - 5623

	65438
10's complement of 05623	<u>+94377</u>
	159815
Discard end carry 10^5	<u>-100000</u>
Answer	59815



SUBTRACTION WITH COMPLEMENTS

- 5623 - 65438

$$\begin{array}{r} 05623 \\ +34562 \\ \hline 40185 \end{array}$$

10's complement of 65438 

There is no
end carry.

Therefore, the answer is:

$$- (10\text{'s complement of } 40185) = - 59815.$$



SUBTRACTION WITH COMPLEMENTS

- 10110010 - 10011111

	10110010
2's complement of 10011111	<u>+01100001</u>
	100010011
Discard end carry 2^8	<u>- 100000000</u>
Answer	00010011



SUBTRACTION WITH COMPLEMENTS

- $10011111 - 10110010$

$$\begin{array}{r} 10011111 \\ 2\text{'s complement of } 10110010 \quad + \underline{01001110} \\ \hline \text{→ } 11101101 \end{array}$$

There is no
end carry.

Therefore, the answer is

$$Y - X = - (2\text{'s complement of } 11101101) = - 00010011.$$



SUBTRACTION WITH COMPLEMENTS

- Subtraction of unsigned numbers can also be done by means of the $(r - 1)$'s complement.
- Remember that the $(r - 1)$'s complement is one less than the r 's complement.
- $10110010 - 10011111$

	10110010
1's complement of 10011111	<u>+01100000</u>
	100010010
End-around carry	<u>+</u> 1
Answer	00010011



SUBTRACTION WITH COMPLEMENTS

- 10011111 - 10110010

$$\begin{array}{r} 10011111 \\ 1\text{'s complement of } 10110010 \quad + \underline{01001101} \\ \hline \text{→ } 11101100 \end{array}$$

There is no
end carry.

Therefore, the answer is

$$Y - X = - (1\text{'s complement of } 11101100) = - 00010011.$$



SIGNED BINARY NUMBERS

- To represent negative integers, we need a notation for negative values
- It is customary to represent the sign with a bit placed in the leftmost position of the number
- The convention is to make the **sign bit 0 for positive** and **1 for negative**
- Example:

Signed-magnitude representation:	10001001
Signed-1's-complement representation:	11110110
Signed-2's-complement representation:	11110111



SIGNED BINARY NUMBERS

ARITHMETIC ADDITION

$$\begin{array}{r}
 + 5 \quad 00000101 \\
 +11 \quad \underline{00001011} \\
 +16 \quad 00010000
 \end{array}$$

$$\begin{array}{r}
 + 5 \quad 00000101 \\
 -11 \quad \underline{11110101} \\
 -6 \quad 11111010
 \end{array}$$

$$\begin{array}{r}
 \text{Discard} \uparrow \\
 - 5 \quad 11111011 \\
 +11 \quad \underline{00001011} \\
 +6 \quad \textcolor{red}{1}00000110
 \end{array}$$

$$\begin{array}{r}
 - 5 \quad 11111011 \\
 -11 \quad \underline{11110101} \\
 -16 \quad \textcolor{red}{1}11110000
 \end{array}$$

 Discard



SIGNED BINARY NUMBERS

ARITHMETIC SUBTRACTION

- In 2's-complement form:

1. Take the 2's complement of the subtrahend (including the sign bit) and add it to the minuend (including sign bit).
2. A carry out of sign-bit position is discarded.



$$(\pm A) - (+B) = (\pm A) + (-B)$$

$$(\pm A) - (-B) = (\pm A) + (+B)$$

- Example:

$$(-6) - (-13) \quad \longrightarrow \quad (11111010 - 11110011)$$

$$\quad \longrightarrow \quad (11111010 + 00001101)$$

$$\quad \longrightarrow \quad 00000111 (+7)$$



BINARY CODE: BCD CODE

- A number with k decimal digits will require $4k$ bits in BCD
- A decimal number in BCD is the same as its equivalent binary number only when the number is between 0 and 9

Table 1.4
Binary-Coded Decimal (BCD)

Decimal Symbol	BCD Digit
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001



BINARY CODE: BCD CODE

- BCD Addition

4	0100	4	0100	8	1000
<u>+5</u>	<u>+0101</u>	<u>+8</u>	<u>+1000</u>	<u>+9</u>	<u>+1001</u>
9	1001	12	1100	17	10001
			<u>+0110</u>		<u>+0110</u>
			10010		10111



BINARY CODE: BCD CODE

- Example:
 - Addition of $184 + 576 = 760$ in BCD

BCD	1	1		
	0001	1000	0100	184
	<u>+ 0101</u>	<u>0111</u>	<u>0110</u>	+576
Binary sum	0111	10000	1010	
Add 6	<u> </u>	<u>0110</u>	<u>0110</u>	<u> </u>
BCD sum	0111	0110	0000	760



BINARY CODE: BCD CODE

- Decimal Arithmetic: $(+375) + (-240) = +135$

0	375
<u>+ 9</u>	<u>760</u>
0	135



BINARY CODES

OTHER DECIMAL CODES

Table 1.5
Four Different Binary Codes for the Decimal Digits

Decimal Digit	BCD 8421	2421	Excess-3	8, 4, -2, -1
0	0000	0000	0011	0000
1	0001	0001	0100	0111
2	0010	0010	0101	0110
3	0011	0011	0110	0101
4	0100	0100	0111	0100
5	0101	1011	1000	1011
6	0110	1100	1001	1010
7	0111	1101	1010	1001
8	1000	1110	1011	1000
9	1001	1111	1100	1111
Unused bit combi- nations	1010	0101	0000	0001
	1011	0110	0001	0010
	1100	0111	0010	0011
	1101	1000	1101	1100
	1110	1001	1110	1101
	1111	1010	1111	1110



BINARY CODE: GRAY CODE

Table 1.6
Gray Code

Gray Code	Decimal Equivalent
0000	0
0001	1
0011	2
0010	3
0110	4
0111	5
0101	6
0100	7
1100	8
1101	9
1111	10
1110	11
1010	12
1011	13
1001	14
1000	15



BINARY CODES: ASCII CHARACTER CODE

- American Standard Code for Information Interchange (Refer to Table 1.7)
- A popular code used to represent information sent as character-based data.
- It uses 7-bits to represent:
 - 94 Graphic printing characters.
 - 34 Non-printing characters.
- Some non-printing characters are used for text format (e.g. BS = Backspace, CR = carriage return).
- Other non-printing characters are used for record marking and flow control (e.g. STX and ETX start and end text areas).



BINARY CODES: ASCII CHARACTER CODE

Table 1.7

American Standard Code for Information Interchange (ASCII)

$b_4b_3b_2b_1$	$b_7b_6b_5$							
	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	“	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	·	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	—	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	—	o	DEL

BINARY CODES: ERROR-DETECTING CODE

- To detect errors in data communication and processing, an eighth bit is sometimes added to the ASCII character to indicate its parity.
- A **parity bit** is an extra bit included with a message to make the total number of 1's either even or odd.
- Example:
 - Consider the following two characters and their even and odd parity:

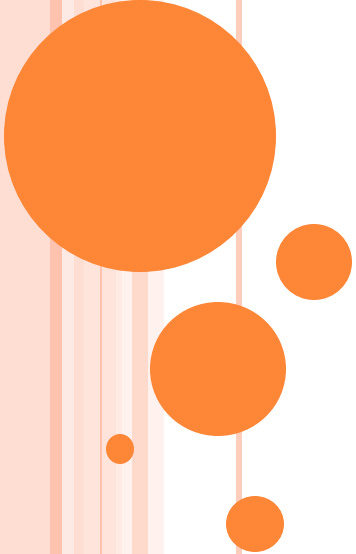
	With even parity	With odd parity
ASCII A = 1000001	01000001	11000001
ASCII T = 1010100	11010100	01010100



SYLLABUS

- Chapter 1 (Excluding Section 1.8)





CSE 205: DIGITAL LOGIC DESIGN

Dr. Tanzima Hashem

Associate Professor

CSE, BUET

LOGIC MINIMIZATION

- The complexity of the digital logic gates
 - The complexity of the algebraic expression
- Algebraic approaches:
 - The procedure of simplifying Boolean expressions is difficult since it lacks specific rules to predict the successive steps in the simplification process.



THE MAP METHOD

- The Karnaugh map (K-Map)
 - A simple straight forward procedure
 - A pictorial form of a truth table
- A diagram made up of squares
 - Each square represents one minterm



TWO-VARIABLE MAP

- Four minterms
- $A' = \text{row } 0$; $A = \text{row } 1$
- $B' = \text{column } 0$; $B = \text{column } 1$

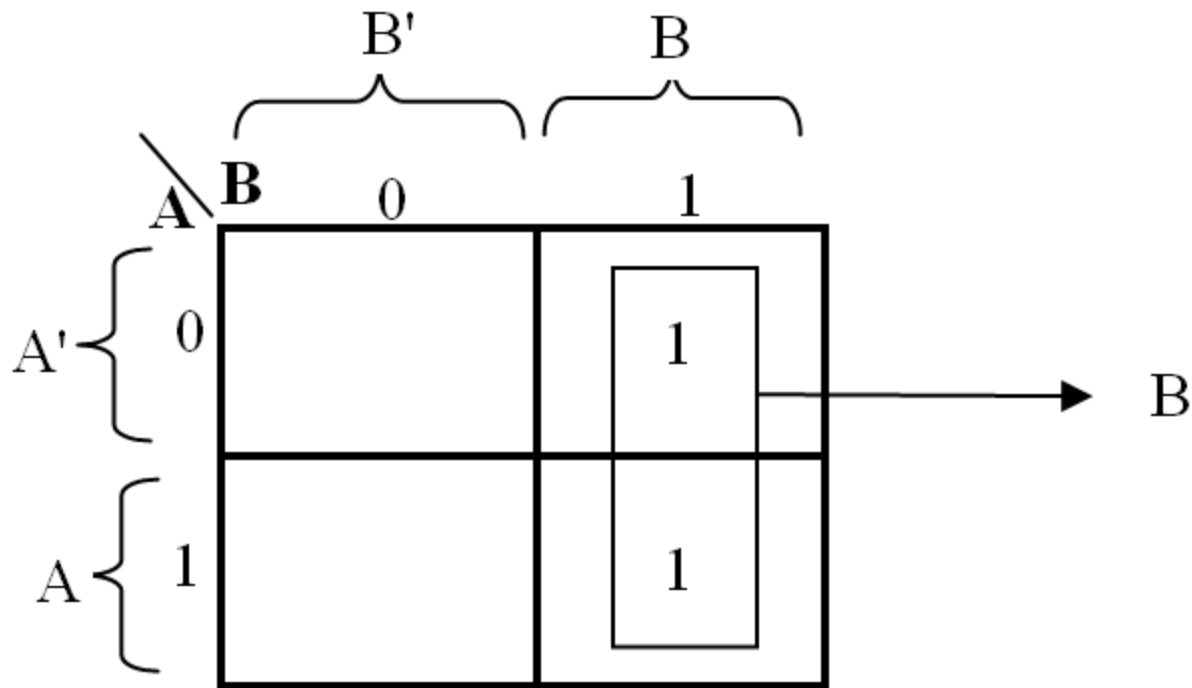
		B'		B	
		B			
$A \backslash B$		0	1		
A'	0	$A'B'$	$A'B$		
	1	AB'	AB		

		B'		B	
		B			
$A \backslash B$		0	1		
A'	0	m_0	m_1		
	1	m_2	m_3		



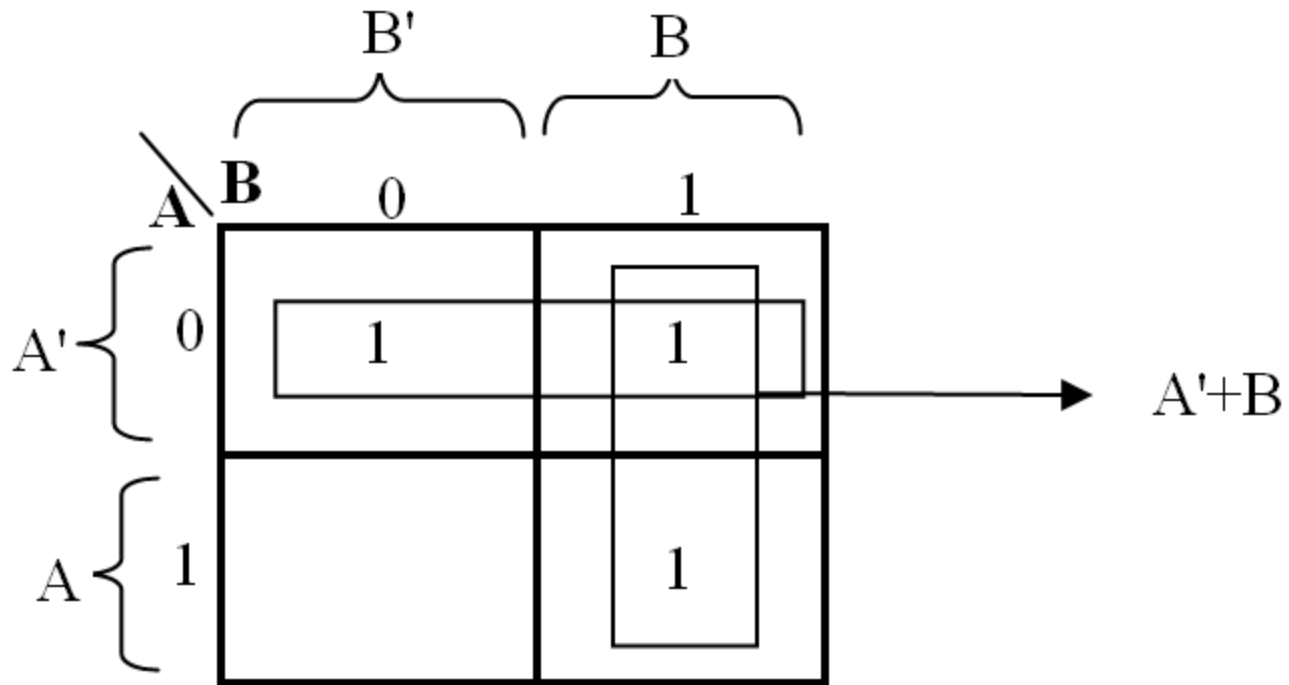
EXAMPLE

$$F = AB + A'B = B$$



EXAMPLE

$$F = AB + A'B + A'B' = A' + B$$



THREE-VARIABLE MAP

- Eight minterms
- The Gray code sequence
- Any two adjacent squares in the map differ by only one variable

		B'		B	
		00	01	11	10
A	0	$A'B'C'$	$A'B'C$	$A'BC$	$A'BC'$
	1	$AB'C'$	$AB'C$	ABC	ABC'



THREE-VARIABLE MAP

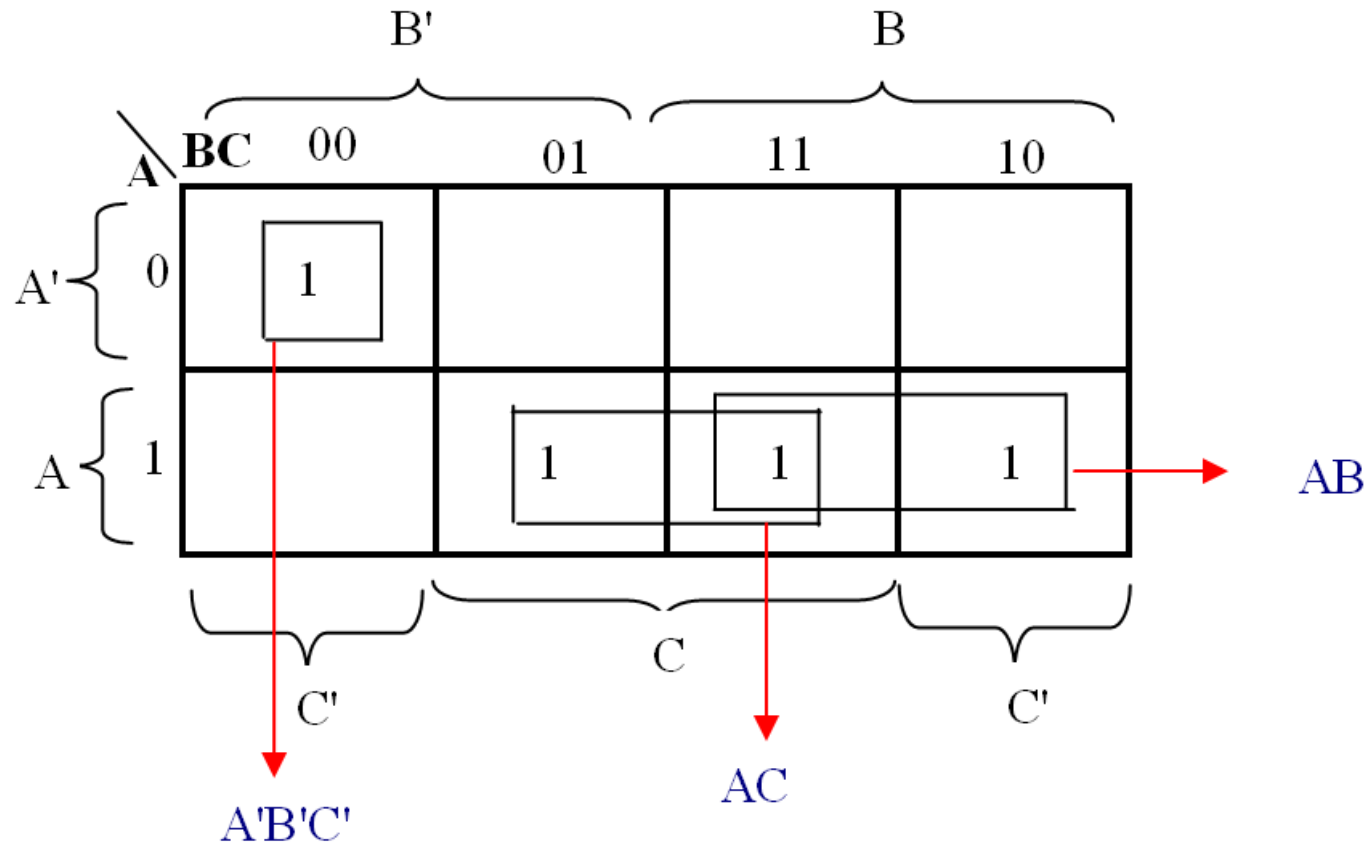
- m_0 and m_2 (m_4 and m_6) are adjacent
- $m_0 + m_2 = A'B'C' + A'BC' = A'C'(B' + B) = A'C'$
- $m_4 + m_6 = AB'C' + A'BC' = AC'(B' + B) = AC'$

		B'		B	
		00	01	11	10
A	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6
		C'		C	C'



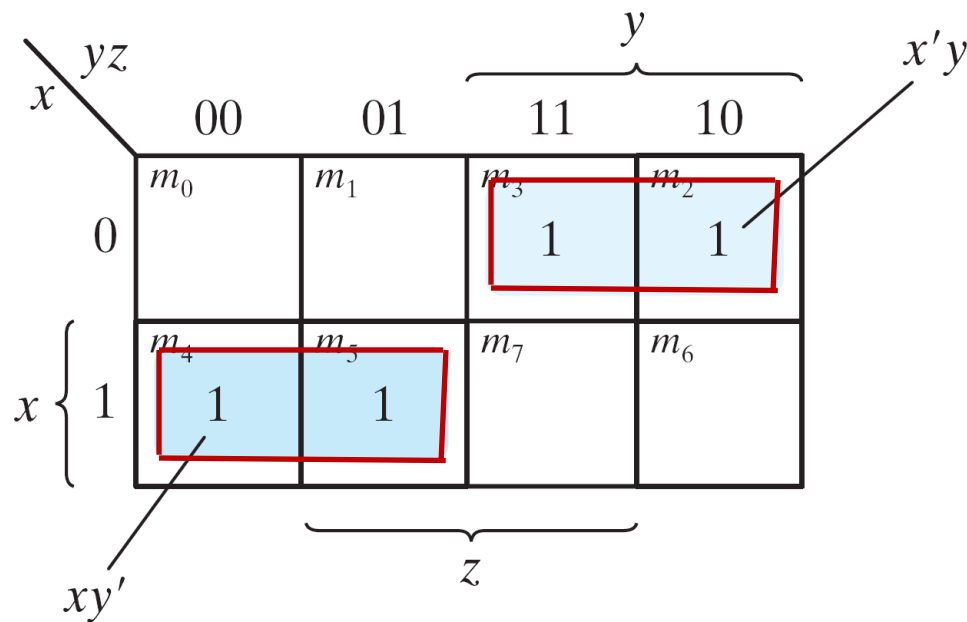
EXAMPLE

$$\begin{aligned} F &= ABC' + A'B'C' + AB'C + ABC \\ &= AB + AC + A'B'C' \end{aligned}$$



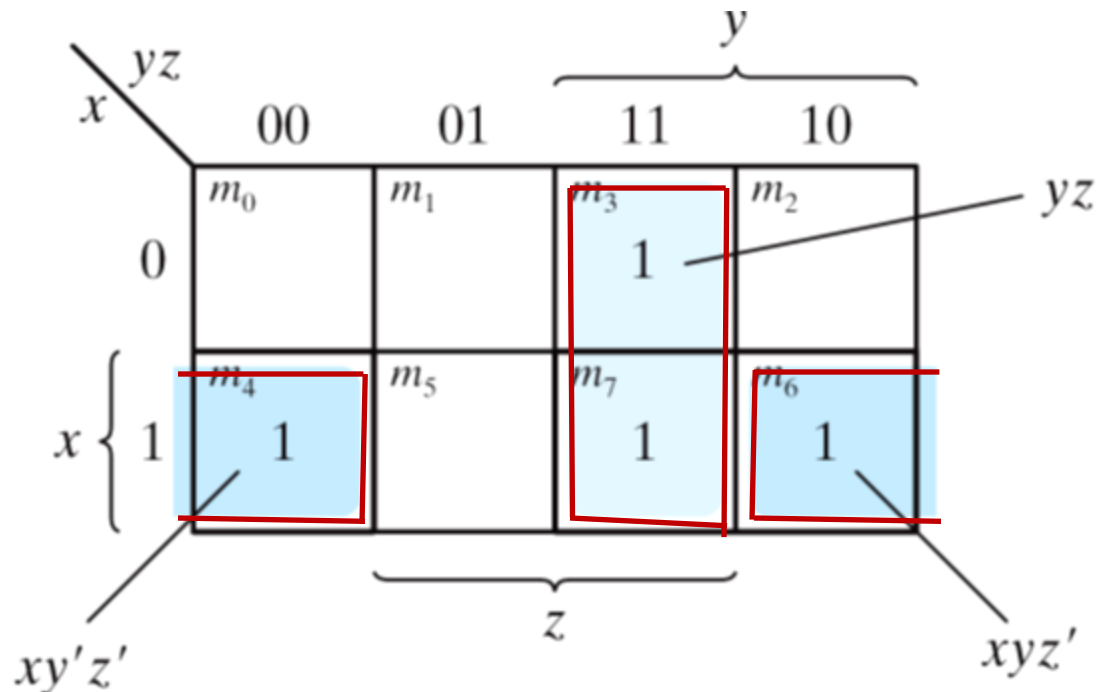
EXAMPLE

- Simplify the Boolean function $F(x, y, z) = \Sigma(2, 3, 4, 5)$
 - $F(x, y, z) = \Sigma(2, 3, 4, 5) = x'y + xy'$



EXAMPLE

- Simplify $F(x, y, z) = \Sigma(3, 4, 6, 7)$
 - $F(x, y, z) = \Sigma(3, 4, 6, 7) = yz + xz'$



Note: $xy'z' + xyz' = xz'$



FOUR ADJACENT SQUARES

○ Consider four adjacent squares

- $m_0 + m_2 + m_4 + m_6 = x'y'z' + x'yz' + xy'z' + xyz' = x'z'(y' + y) + xz'(y' + y) = x'z' + xz' = z'$
- $m_1 + m_3 + m_5 + m_7 = x'y'z + x'yz + xy'z + xyz = x'z(y' + y) + xz(y' + y) = x'z + xz = z$

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

(a)

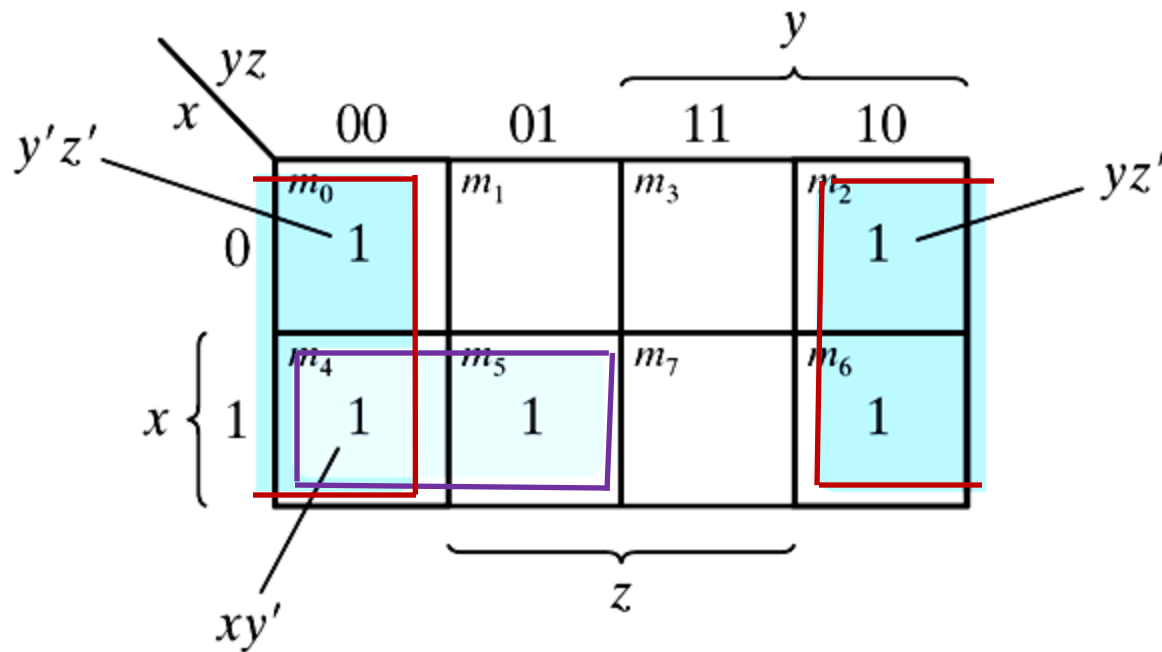
		y			
		xz			
		00	01	11	10
x	0	$x'y'z'$	$x'y'z$	$x'yz$	$x'yz'$
	1	$xy'z'$	$xy'z$	xyz	xyz'
		z			

(b)



EXAMPLE

- Simplify $F(x, y, z) = \Sigma(0, 2, 4, 5, 6)$
 - $F(x, y, z) = \Sigma(0, 2, 4, 5, 6) = z' + xy'$

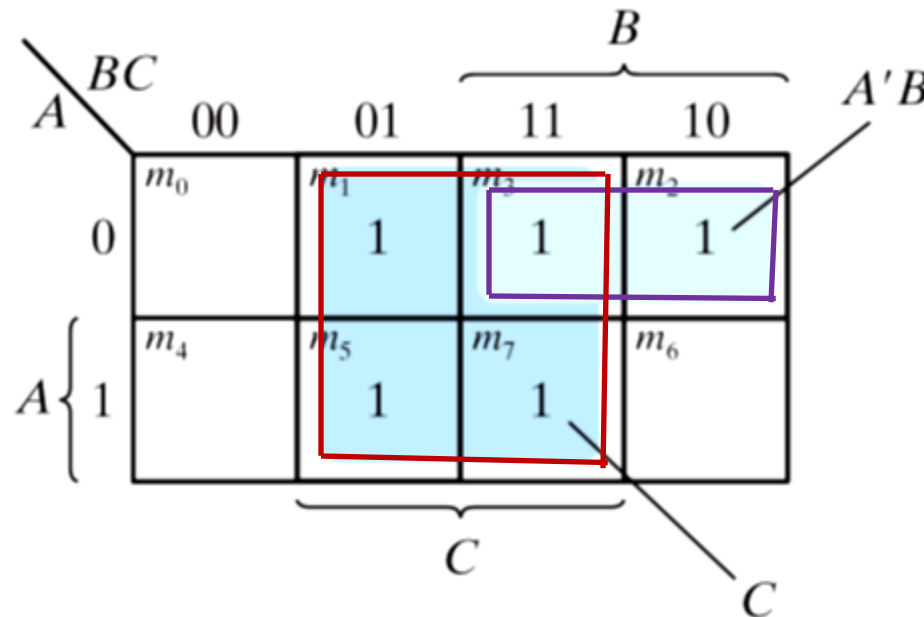


Note: $y'z' + yz' = z'$



EXAMPLE

- Let $F = A'C + A'B + AB'C + BC$
 - Express it in sum of minterms.
 - Find the minimal sum of products expression.
$$F(A, B, C) = \Sigma(1, 2, 3, 5, 7) = C + A'B$$



FOUR-VARIABLE MAP

- 16 minterms
- Combinations of 2, 4, 8, and 16 adjacent squares

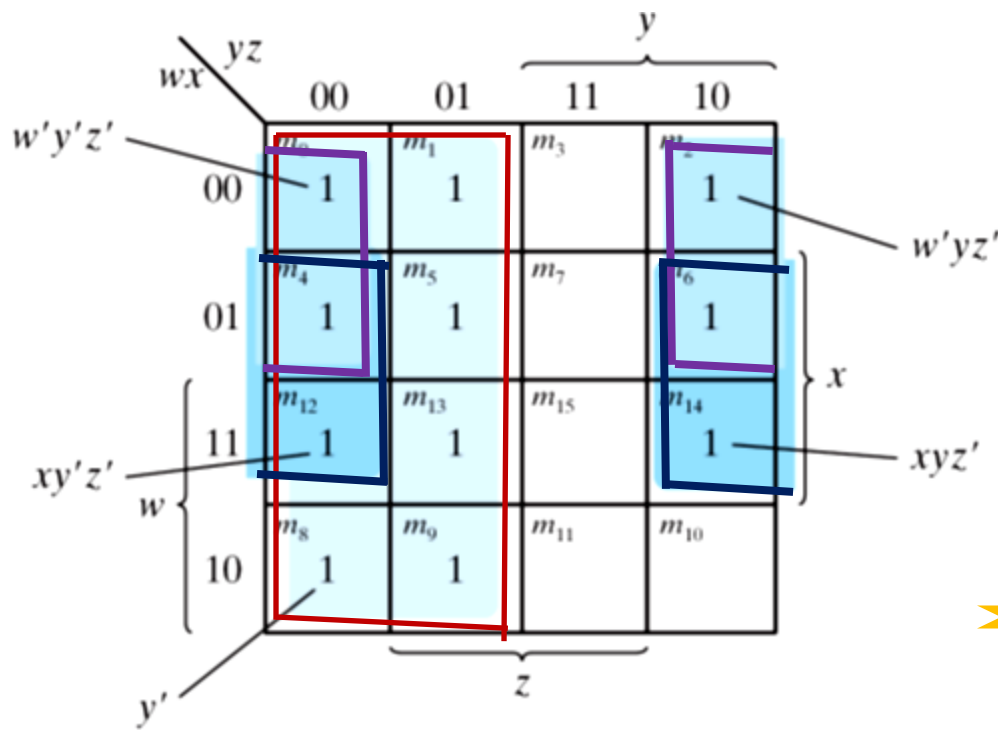
		C'		C			
AB		CD	00	01	11	10	
A'	00	$A' B' C' D'$	$A' B' C' D$	$A' B' C D$	$A' B' C D'$	A'	
	01	$A' B C' D'$	$A' B C' D$	$A' B C D$	$A' B C D'$		
A	11	$A B C' D'$	$A B C' D$	$A B C D$	$A B C D'$	A	
	10	$A B' C' D'$	$A B' C' D$	$A B' C D$	$A B' C D'$		
		D'		D		D'	

		C'		C		
AB		CD	00	01	11	10
A'	00	m_0	m_1	m_3	m_2	B'
	01	m_4	m_5	m_7	m_6	B
A	11	m_{12}	m_{13}	m_{15}	m_{14}	B'
	10	m_8	m_9	m_{11}	m_{10}	
		D'		D		D'



EXAMPLE 3.5

- Example 3.5: simplify $F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$



Note: $w'y'z' + w'yz' = w'z'$
 $xy'z' + xyz' = xz'$

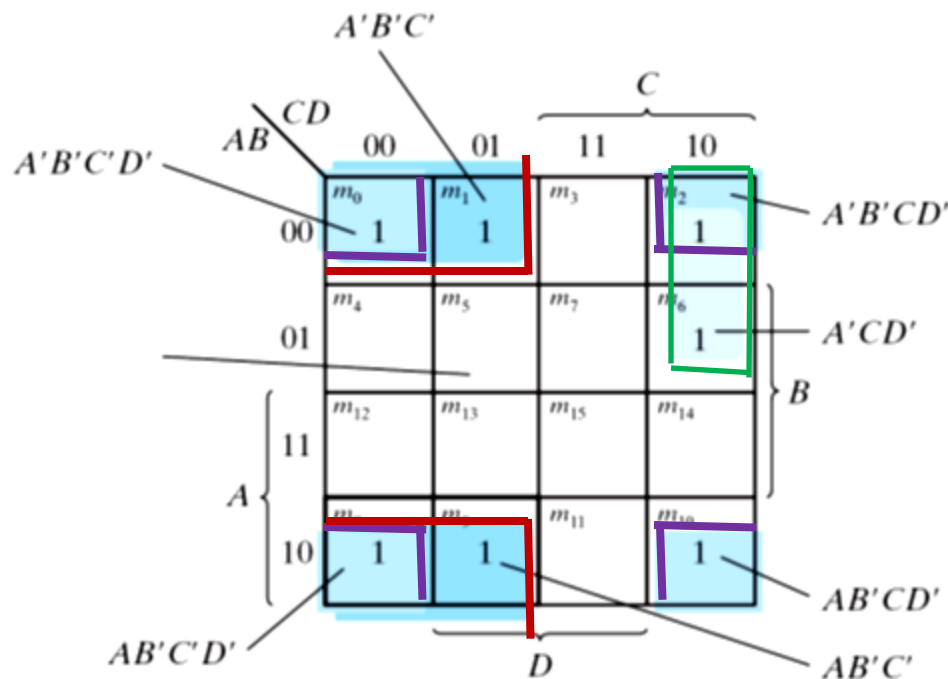


$$F = y' + w'z' + xz'$$



EXAMPLE

- Simplify $F = A'B'C' + B'CD' + A'B'CD' + AB'C'$



Note: $A'B'C'D' + A'B'CD' = A'B'D'$
 $AB'C'D' + AB'CD' = AB'D'$
 $A'B'D' + AB'D' = B'D'$
 $A'B'C' + AB'C' = B'C'$

$$A'B'C' + B'CD' + A'B'CD' + AB'C' = B'D' + B'C' + A'CD'$$



PRIME IMPLICANTS

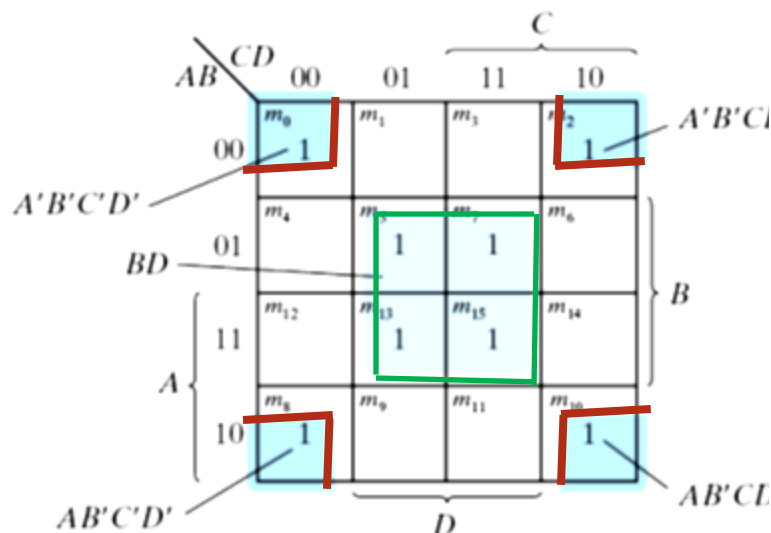
- All the minterms are covered.
- Minimize the number of terms.
- There are no redundant terms.

- Implicant: A group of one or more k map cell
- Prime implicant: an implicant that is not a subset of another implicant
- Essential Prime Implicant: a prime implicant that covers at least one cell not covered by another prime implicant



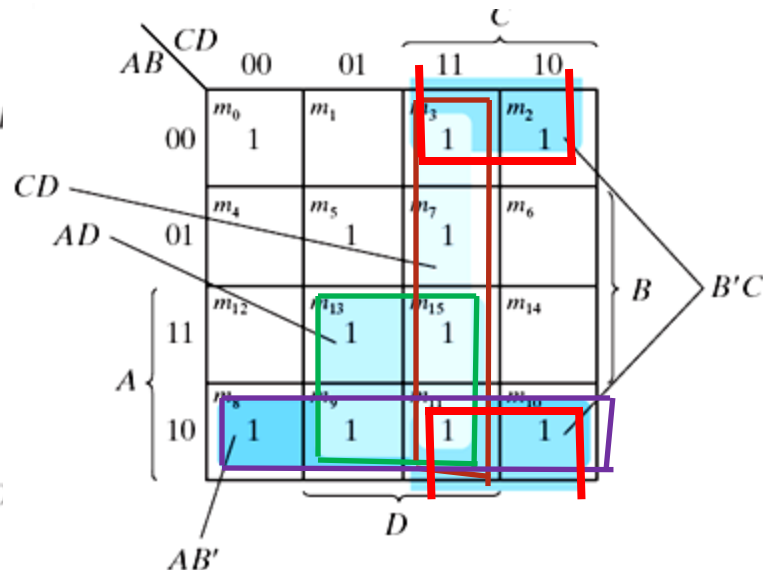
PRIME IMPLICANTS

- $F(A, B, C, D) = \Sigma(0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$
 - The simplified expression may not be unique
 - $F = BD + B'D' + CD + AD = BD + B'D' + CD + AB'$
 $= BD + B'D' + B'C + AD = BD + B'D' + B'C + AB'$



Note: $A'B'C'D' + A'B'CD' = A'B'D'$
 $AB'C'D' + AB'CD' = AB'D'$
 $A'B'D' + AB'D' = B'D'$

(a) Essential prime implicants
 BD and $B'D'$



(b) Prime implicants CD , $B'C$,
 AD , and AB'

FIVE-VARIABLE MAP

- Map for more than four variables becomes complicated
 - Five-variable map: two four-variable map (one on the top of the other).

$A = 0$				
DE		D		
BC	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10
E				

B is indicated by a bracket on the left side of the map, and C is indicated by a bracket on the right side of the map.

$A = 1$				
DE		D		
BC	00	01	11	10
00	16	17	19	18
01	20	21	23	22
11	28	29	31	30
10	24	25	27	26
E				

B is indicated by a bracket on the left side of the map, and C is indicated by a bracket on the right side of the map.



EXAMPLE

- Simplify $F = \Sigma(0, 2, 4, 6, 9, 13, 21, 23, 25, 29, 31)$

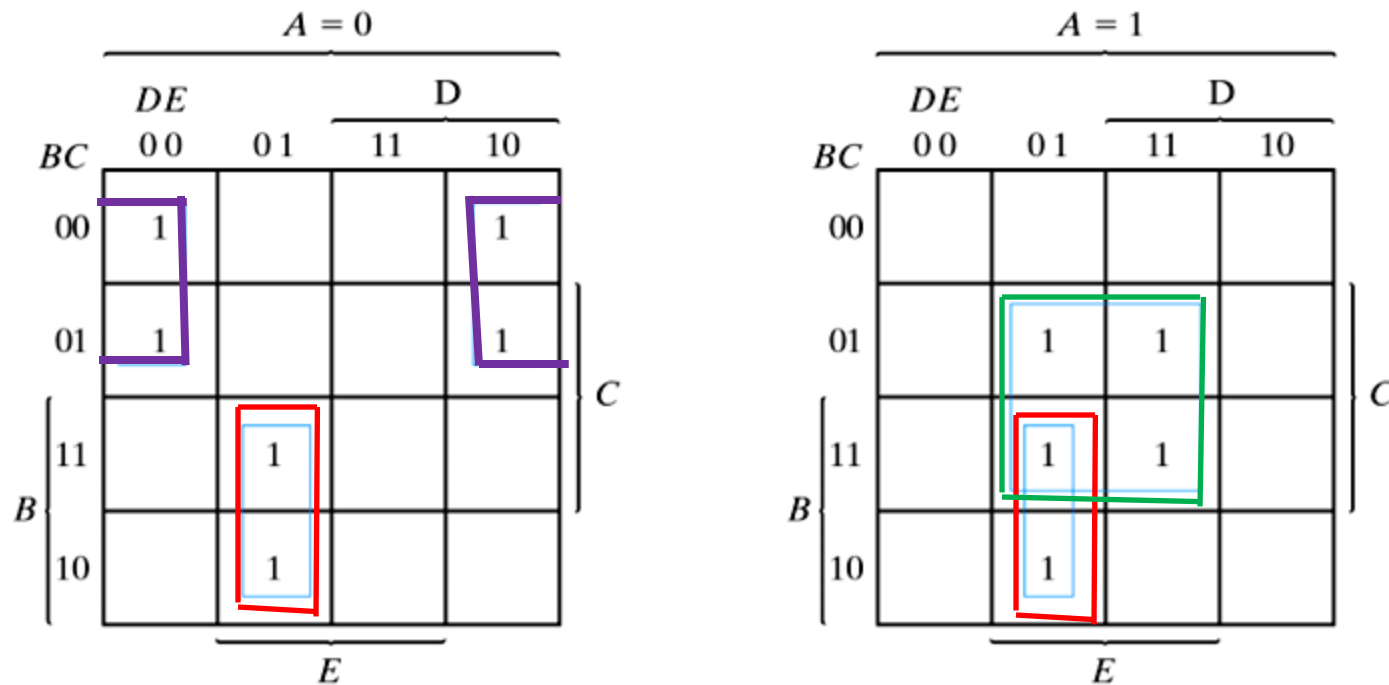


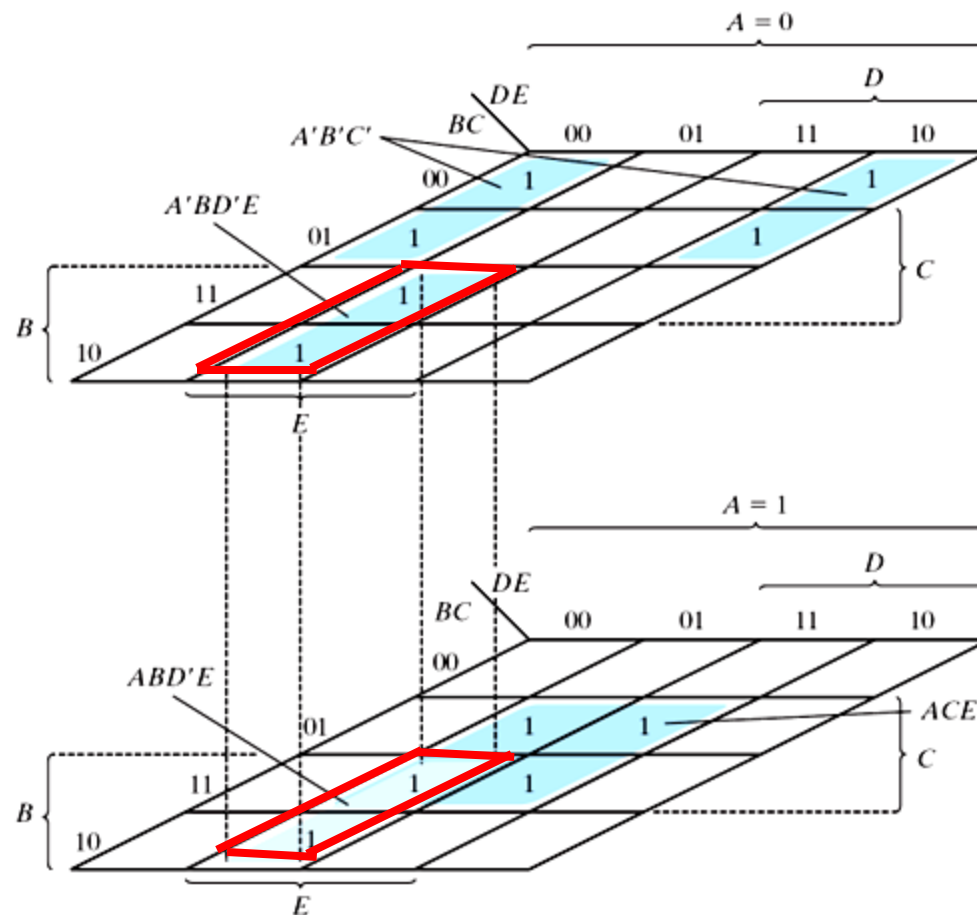
Fig. 3-13 Map for Example 3-7; $F = A'B'E' + BD'E + ACE$

➡ $F = A'B'E' + BD'E + ACE$



EXAMPLE

- Another Map for $F = A'B'E' + BD'E + ACE$



SIX-VARIABLE MAP

- Another Map for $F = A'B'E' + BD'E + ACE$

		$v = 0$				$v = 1$				
		zw								
		xy	00	01	11	10	00	01	11	10
$u = 0$	{	00	m_0	m_1	m_3	m_2	m_{16}	m_{17}	m_{19}	m_{18}
		01	m_4	m_5	m_7	m_6	m_{20}	m_{21}	m_{23}	m_{22}
		11	m_{12}	m_{13}	m_{15}	m_{14}	m_{28}	m_{29}	m_{31}	m_{30}
		10	m_8	m_9	m_{11}	m_{10}	m_{24}	m_{25}	m_{27}	m_{26}
$u = 1$	{	00	m_{32}	m_{33}	m_{35}	m_{34}	m_{48}	m_{49}	m_{51}	m_{50}
		01	m_{36}	m_{37}	m_{39}	m_{38}	m_{52}	m_{53}	m_{55}	m_{54}
		11	m_{44}	m_{45}	m_{47}	m_{46}	m_{60}	m_{61}	m_{63}	m_{62}
		10	m_{40}	m_{41}	m_{43}	m_{42}	m_{56}	m_{57}	m_{59}	m_{58}
			$v = 0$				$v = 1$			

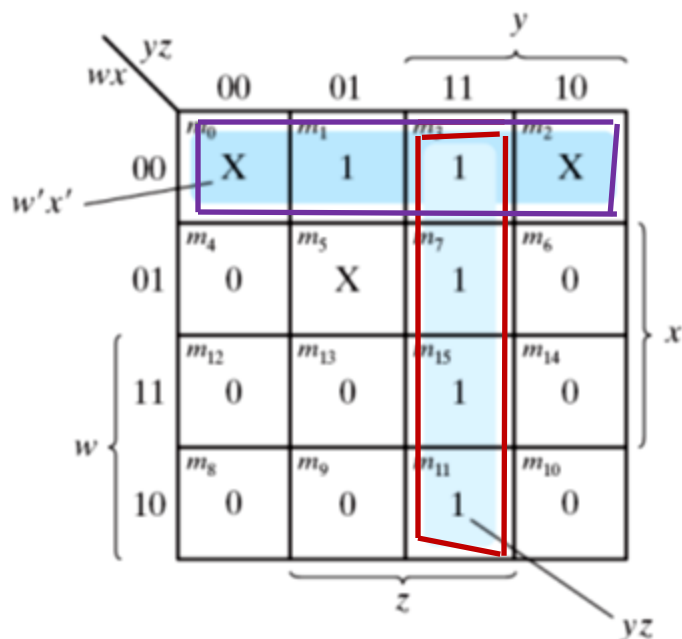
DON'T-CARE CONDITIONS

- The value of a function is not specified for certain combinations of variables
 - BCD; 1010-1111
- The unspecified minterms of a function are called the don't-care conditions, or simply the don't-cares, and are denoted as Xs.
- These don't-care conditions can be used on a map to provide further simplification of the Boolean expression.
 - Can be implemented as 0 or 1

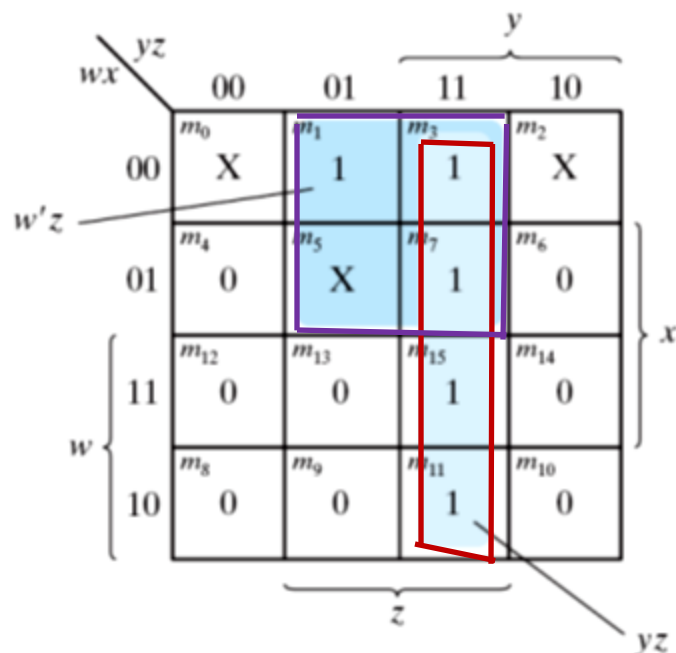


EXAMPLE

- Simplify $F(w, x, y, z) = \Sigma(1, 3, 7, 11, 15)$ which has the don't-care conditions $d(w, x, y, z) = \Sigma(0, 2, 5)$.
- $F = yz + w'x'$; $F = yz + w'z$
- $F = \Sigma(0, 1, 2, 3, 7, 11, 15)$; $F = \Sigma(1, 3, 5, 7, 11, 15)$



(a) $F = yz + w'x'$



(b) $F = yz + w'z$



DON'T-CARE CONDITIONS

- Either one of the above expressions satisfies the conditions stated.
- Note that the above 2 expressions represent 2 functions that are algebraically unequal: each covers different don't-care terms.
- We may or may not include any of the Xs, while all the 1s must be included.



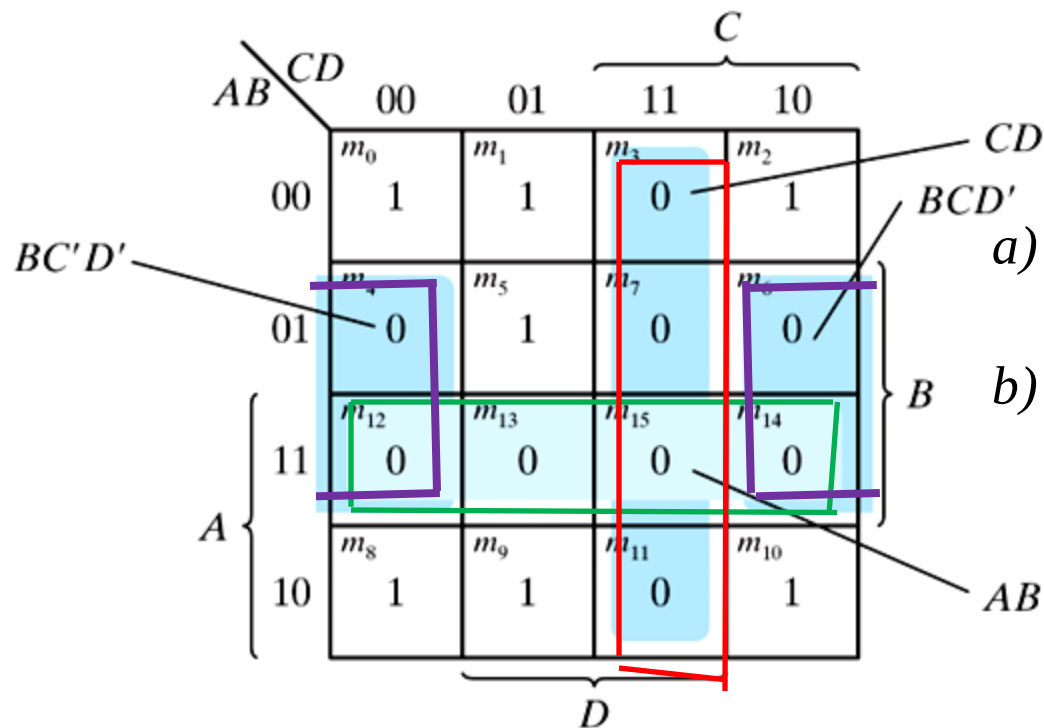
PRODUCT OF SUMS SIMPLIFICATION

- Simplify F' in the form of sum of products
- Apply DeMorgan's theorem $F = (F')'$
- F' : sum of products $\rightarrow F$: product of sums



EXAMPLE

- Simplify $F = \Sigma(0, 1, 2, 5, 8, 9, 10)$ into (a) sum-of-products form, and (b) product-of-sums form:

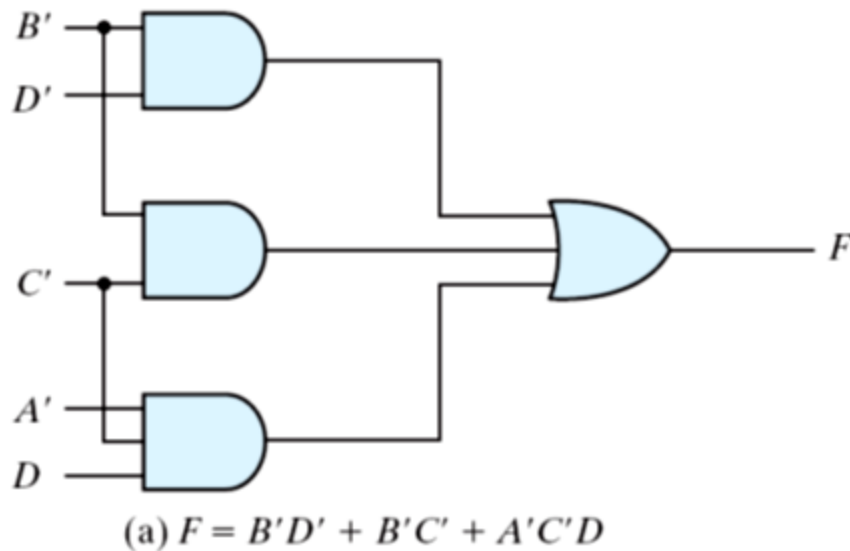


Note: $BC'D' + BCD' = BD'$

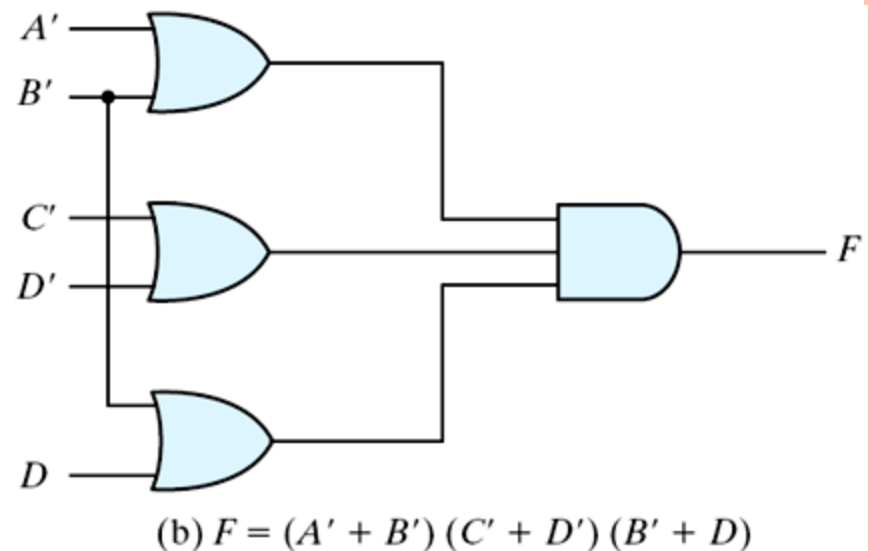
- a) $F(A, B, C, D) = \Sigma(0, 1, 2, 5, 8, 9, 10) = B'D' + B'C' + A'C'D$
- b) $F' = AB + CD + BD'$
- » Apply DeMorgan's theorem;
 $F = (A' + B')(C' + D')(B' + D)$

EXAMPLE (CONT.)

- Gate implementation of $F = \Sigma(0, 1, 2, 5, 8, 9, 10)$



Sum-of products form



Product-of sums form

Figure 3.15 Gate Implementation of $F = \Sigma(0, 1, 2, 5, 8, 9, 10)$

PRODUCT OF MAXTERMS

- Consider the function defined in Table 3.2.

- In sum-of-minterm:

$$F(x, y, z) = \sum (1, 3, 4, 6)$$

- In sum-of-maxterm:

$$F(x, y, z) = \prod (0, 2, 5, 7)$$

Table 3.2

Truth Table of Function F

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0



PRODUCT OF MAXTERMS

- Consider the function defined in Table 3.2.

- Combine the 1's:

$$F(x, y, z) = x'z + xz'$$

- Combine the 0's :

$$F'(x, y, z) = xz + x'z'$$

- Taking the complement of F'

$$F(x, y, z) = (x' + z')(x + z)$$

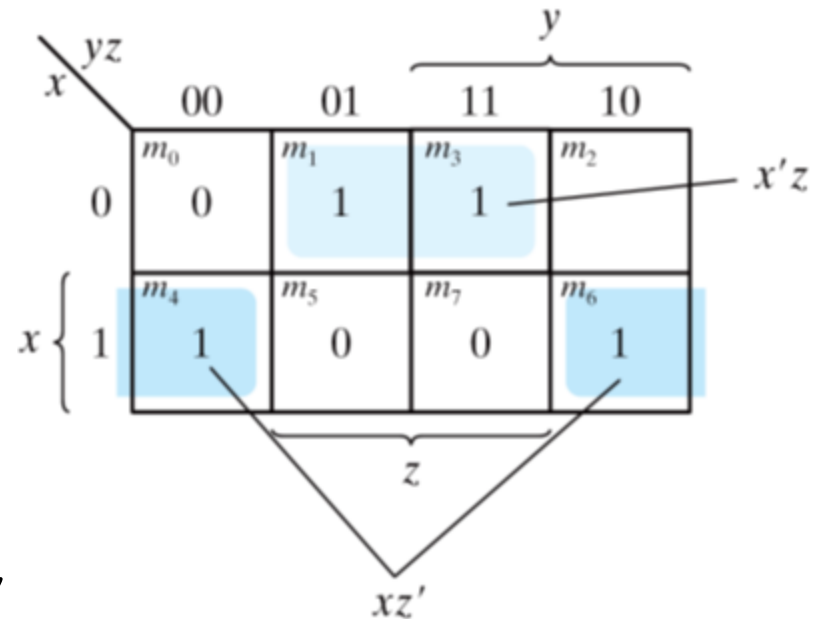


Figure 3.16 Map for the function of Table 3.2

MAP METHOD: SUMMARY

- Simplification procedure:
 - Generate K-map.
 - Determine PIs.
 - Select EPIs.
 - Find minimal cover (set) of PIs.



SIMPLIFICATION OF BOOLEAN FUNCTIONS

- 3 common methods:
- Boolean algebra
 - The basis for all methods
 - Difficult to see the best path to take and to know when you are finished
- Karnaugh maps
 - Fast and easy for 2 – 6 variables
 - Pictorial approach based on recognition of patterns
 - Difficult for large numbers of variables
- Tabulation methods
 - Tedious to perform by hand, but well suited to computer implementation
 - No limit on the number of variables



TABULATION METHODS

- The tabulation method was first formulated by Quine and later improved by McCluskey. It is also known as the **Quine-McCluskey** method.
- Simplification consists of two parts:
 - To find candidate prime implicants.
 - To choose the prime implicants that give an expression with the least number of literals (i.e., minimal cover generation).



DETERMINATION OF PRIME IMPLICANTS

- Group minterms by number of 1s.
- Compare minterms and find pairs with distance 1.
- Generate subcubes.
- Repeat the above procedure on generated subcubes until no more subcubes can be generated.



DETERMINATION OF PRIME IMPLICANTS

- $F(w,x,y,z) = \sum(0, 1, 2, 8, 10, 11, 14, 15)$
 $= w'x'y' + x'z' + wy$

(a)			(b)			(c)		
	$wxyz$			$wxyz$			$wxyz$	
0	0000	✓	0,1	000–		0,2,8,10	–0–0	
			0,2	00–0	✓	0,8,2,10	–0–0	
1	0001	✓	0,8	–000	✓			
2	0010	✓				10,11,14,15	1–1–	
8	1000	✓	2,10	–010	✓	10,14,11,15	1–1–	
			8,10	10–0	✓			
10	1010	✓						
			10,11	101–	✓			
11	1011	✓	10,14	1–10	✓			
14	1110	✓						
			11,15	1–11	✓			
15	1111	✓	14,15	111–	✓			



DETERMINATION OF PRIME IMPLICANTS

- Step 1: Group the minterms according to the number of 1s (see Column (a)).
- Step 2: Combine any 2 minterms that differ from each other by exactly one variable (i.e., dist-1), the unmatched variable removed (see Column (b)). Try this for all possible pairs of minterms. A check (3) is placed to the right of both minterms if they have been used in a match.



DETERMINATION OF PRIME IMPLICANTS

- **Step 3:** Repeat the process. Combine any 2 product terms from Step that differ from each other by exactly one variable, the unmatched variable removed (see Column (c)).
- **Step 4:** The unchecked terms in the table form the PIs. Some of the product terms may appear twice in the table. It of course is unnecessary to use the same term twice.



MINIMAL COVER GENERATION

- The selection of PIs that form the minimized function is made from a PI table
- $F(w,x,y,z) = \sum(1, 4, 6, 7, 8, 9, 10, 11, 15)$

	PI	Minterms	1	4	6	7	8	9	10	11	15
✓	$x'y'z$	1,9	X					X			
✓	$w'xz'$	4,6		X	X						
	$w'xy$	6,7			X	X					
	xyz	7,15				X					X
	wyz	11,15								X	X
✓	wx'	8,9,10,11					X	X	X	X	
			✓	✓	✓		✓	✓	✓	✓	



MINIMAL COVER GENERATION

- $F(w,x,y,z) = \sum(1, 4, 6, 7, 8, 9, 10, 11, 15)$
- Essential Prime implicants are $x'y'z$, $w'xz'$, wx'
- $F(w,x,y,z) = x'y'z + w'xz' + wx' + xyz$

PI	Minterms	7	15
$w'xy$	6, 7	X	
xyz	7, 15	X	X
wyz	11, 15		X



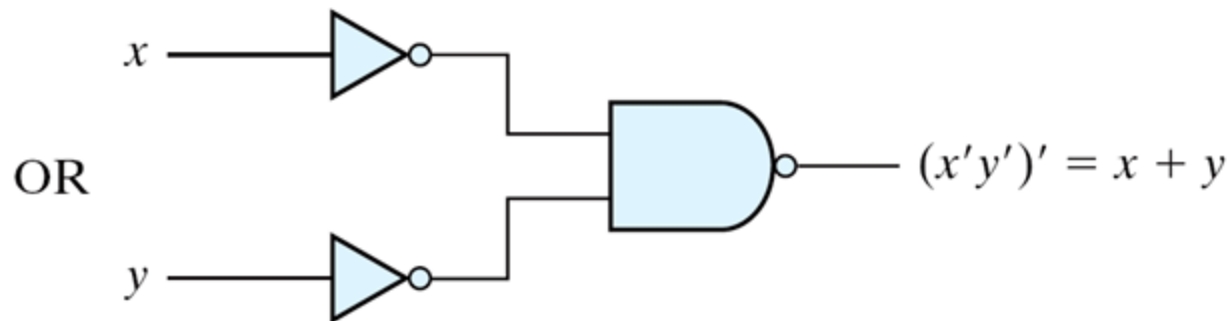
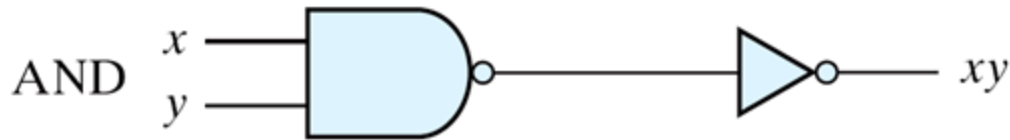
MINIMAL COVER GENERATION

- Step 1: Check the PIs that cover minterms with a single X in their columns. These PIs are the EPIs.
- Step 2: Check each column whose minterm is covered by the selected EPIs.
- Step 3: If there are minterms left uncovered (7 & 15 in this example), some nonessential PIs have to be selected to cover them. We of course will select the PIs with a smallest total number of literals (xyz in this example).



NAND AND NOR IMPLEMENTATION

- NAND gate is a universal gate
 - Can implement any digital system

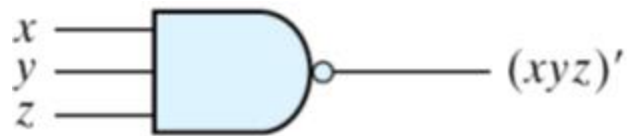


Logic Operations with NAND Gates

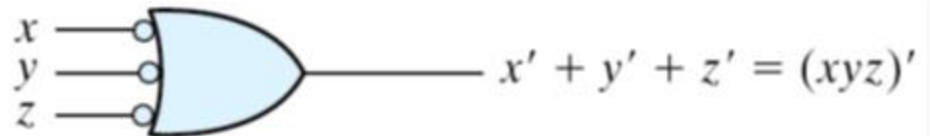


NAND GATE

- Two graphic symbols for a NAND gate



(a) AND-invert



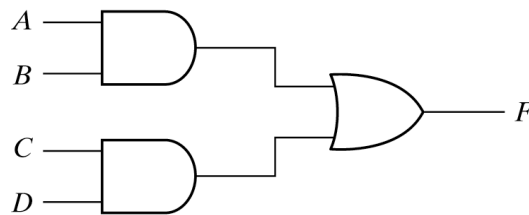
(b) Invert-OR

Two Graphic Symbols for NAND Gate

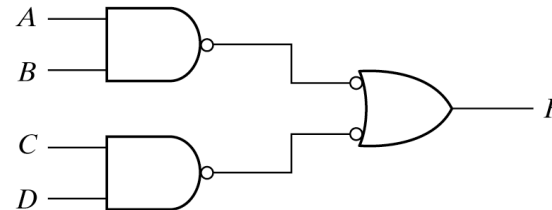


TWO-LEVEL IMPLEMENTATION

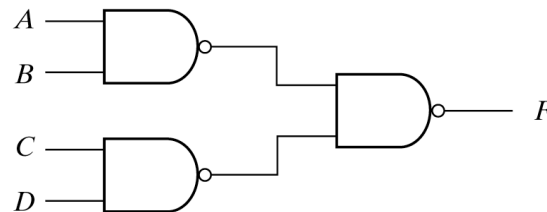
- Two-level logic
 - NAND-NAND = sum of products
 - Example: $F = AB + CD$
 - $F = ((AB)' (CD)')' = AB + CD$



(a)



(b)

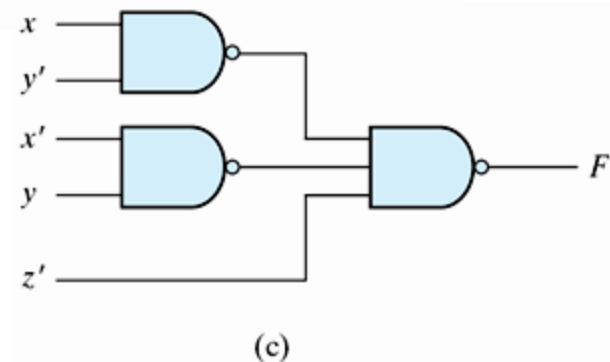
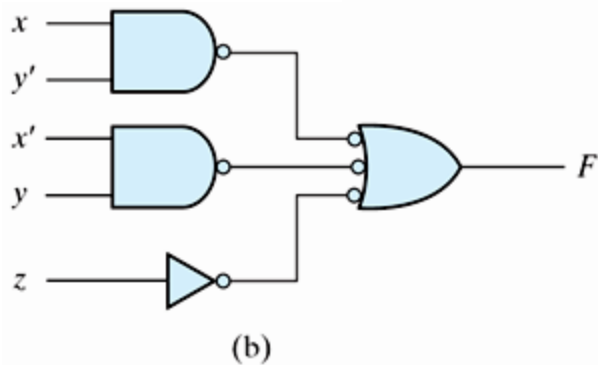
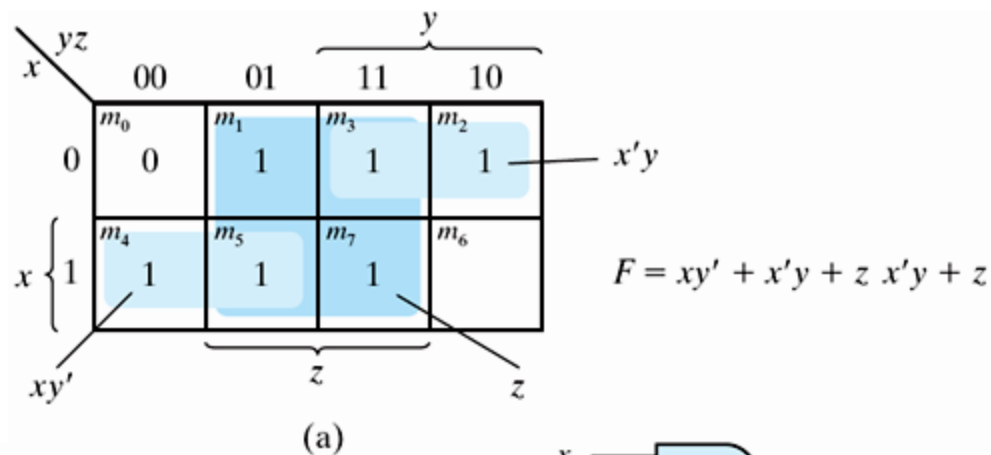


(c)

Fig. 3-20 Three Ways to Implement $F = AB + CD$

EXAMPLE

$$F(x, y, z) = \sum (1, 2, 3, 4, 5, 7) \longrightarrow F(x, y, z) = xy' + x'y + z$$



TWO -LEVEL NAND CIRCUITS

- The procedure
 - Simplify the function and express it in SOP form
 - Draw a NAND gate for each product term
 - Draw a single gate using AND-invert or invert-OR symbol for the sum term
 - A term with single literal, complement if needed



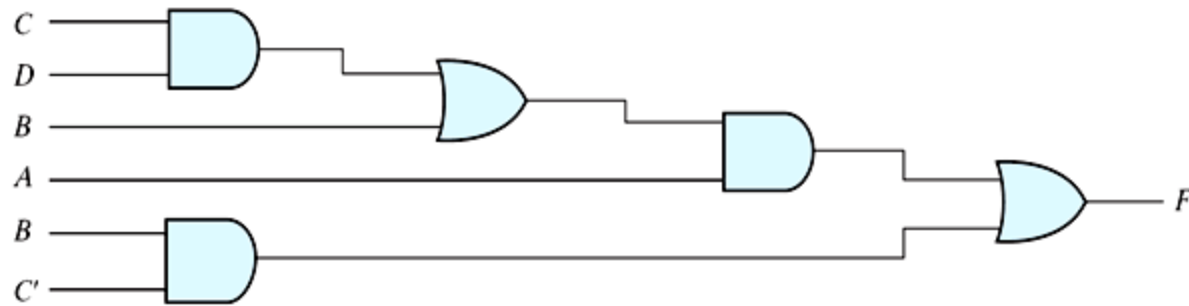
MULTILEVEL NAND CIRCUITS

- The procedure
 - Convert all AND gates to NAND gates
 - Convert all OR gates to NAND gates with invert-OR symbols
 - Balance all bubbles, insert an inverter if needed

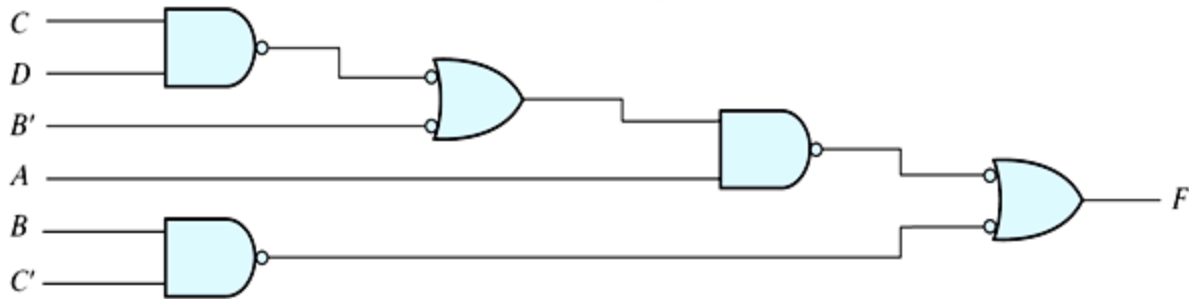


MULTILEVEL NAND CIRCUITS

○ $F = A(CD + B) + BC'$



(a) AND-OR gates

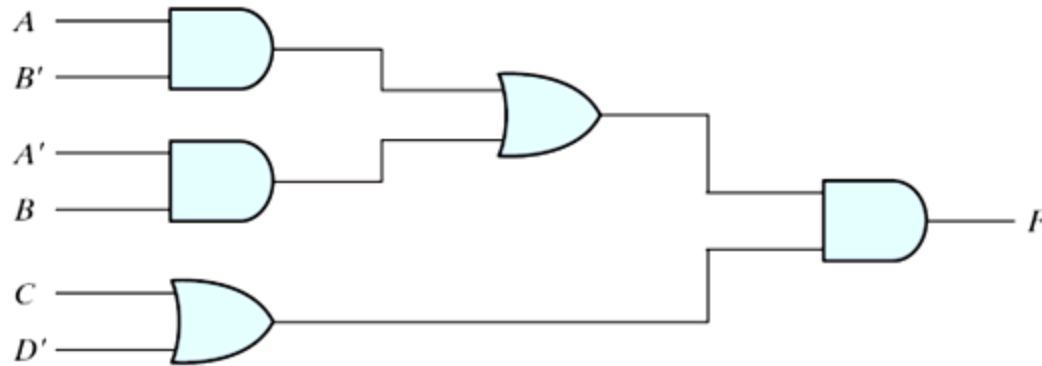


(b) NAND gates

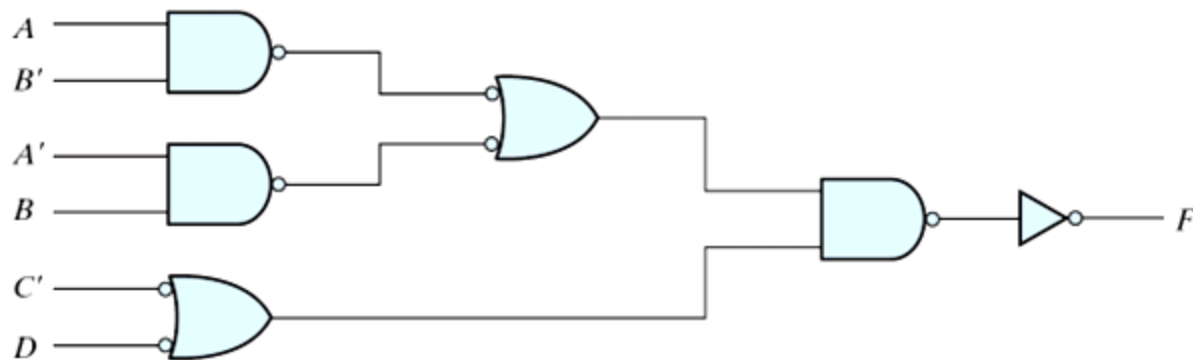


MULTILEVEL NAND CIRCUITS

○ $F = (AB' + A'B)(C + D')$



(a) AND-OR gates

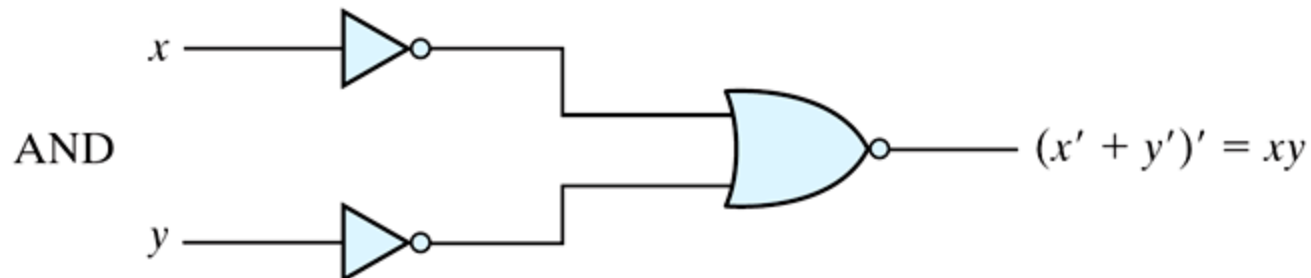
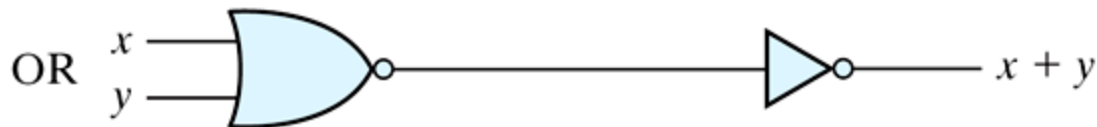


(b) NAND gates



NOR IMPLEMENTATION

- NOR function is the dual of NAND function.
- The NOR gate is also universal.

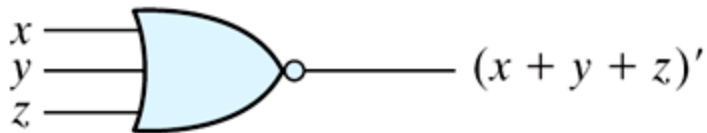


Logic Operation with NOR Gates

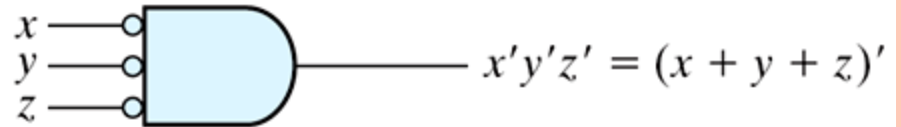


NOR GATE

- Two graphic symbols for a NOR gate



(a) OR-invert



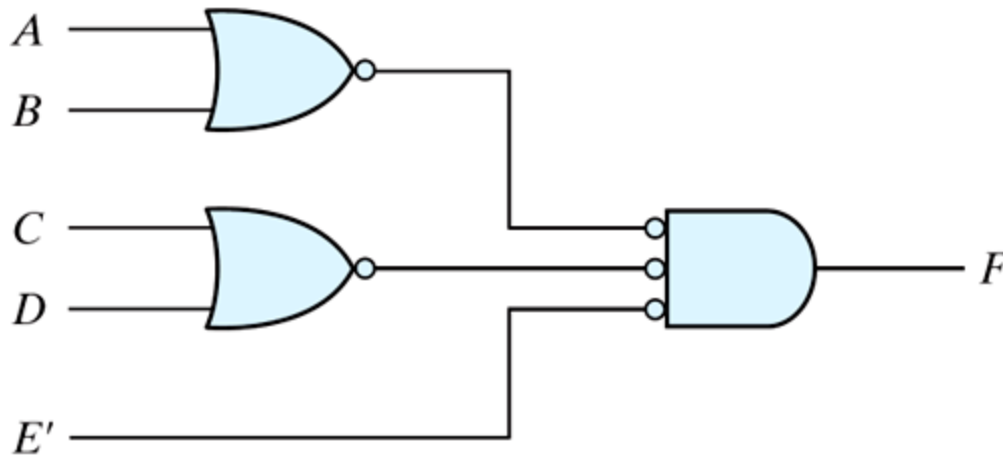
(b) Invert-AND

Two Graphic Symbols for NOR Gate



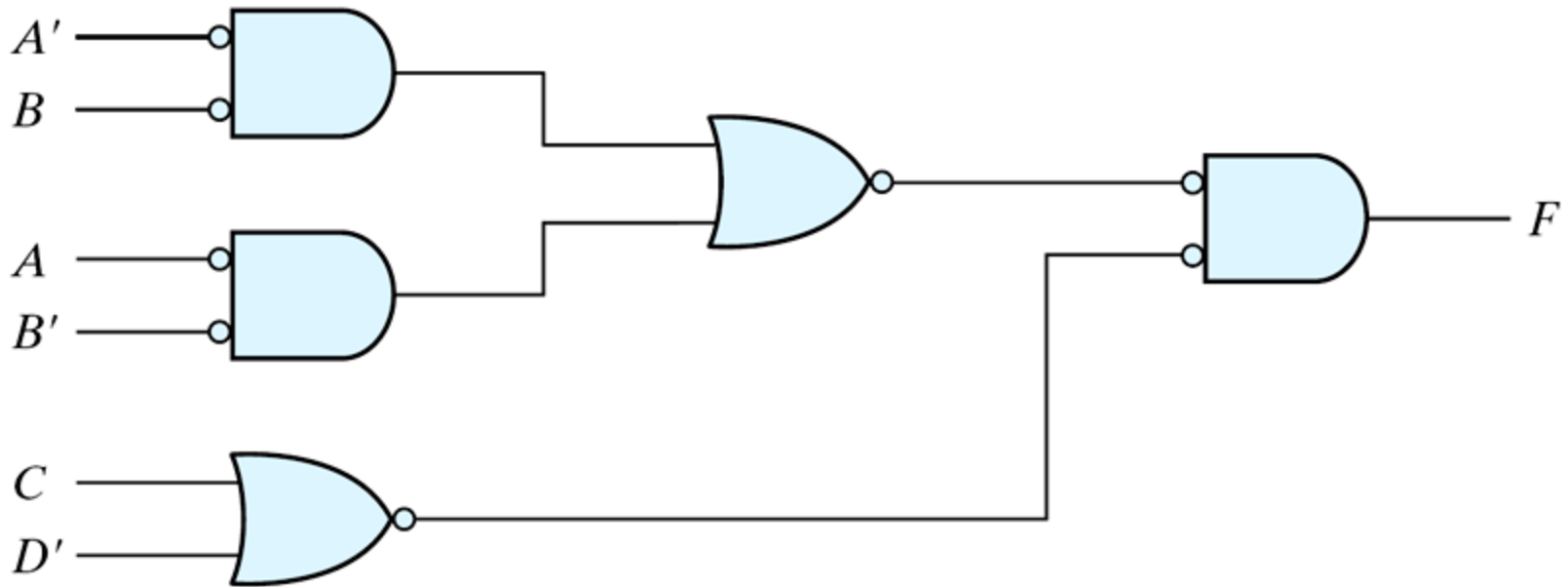
NOR IMPLEMENTATION

- $F = (A + B)(C + D)E$



NOR IMPLEMENTATION

○ $F = (AB' + A'B)(C + D')$



OTHER TWO-LEVEL IMPLEMENTATIONS

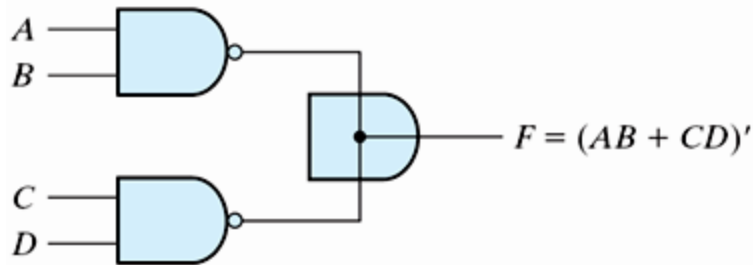
- Wired logic
 - In some technologies, it is possible to short O/Ps of some logic gates

$$F = (AB)' \cdot (CD)' = (AB + CD)' = (A' + B')(C' + D')$$

$$F = (A + B)' + (C + D)' = [(A + B)(C + D)]'$$

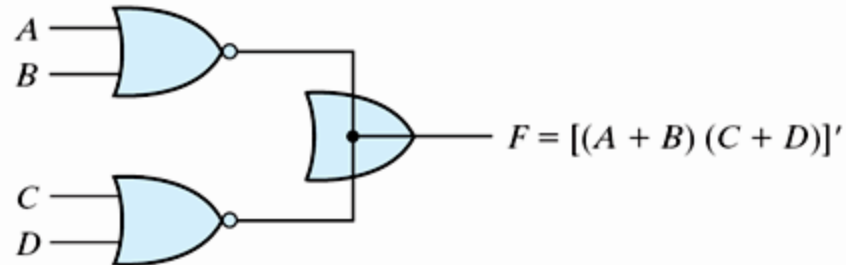
AND-OR-INVERT function

OR-AND-INVERT function



(a) Wired-AND in open-collector
TTL NAND gates.

(AND-OR-INVERT)



(b) Wired-OR in ECL gates

(OR-AND-INVERT)

Wired Logic

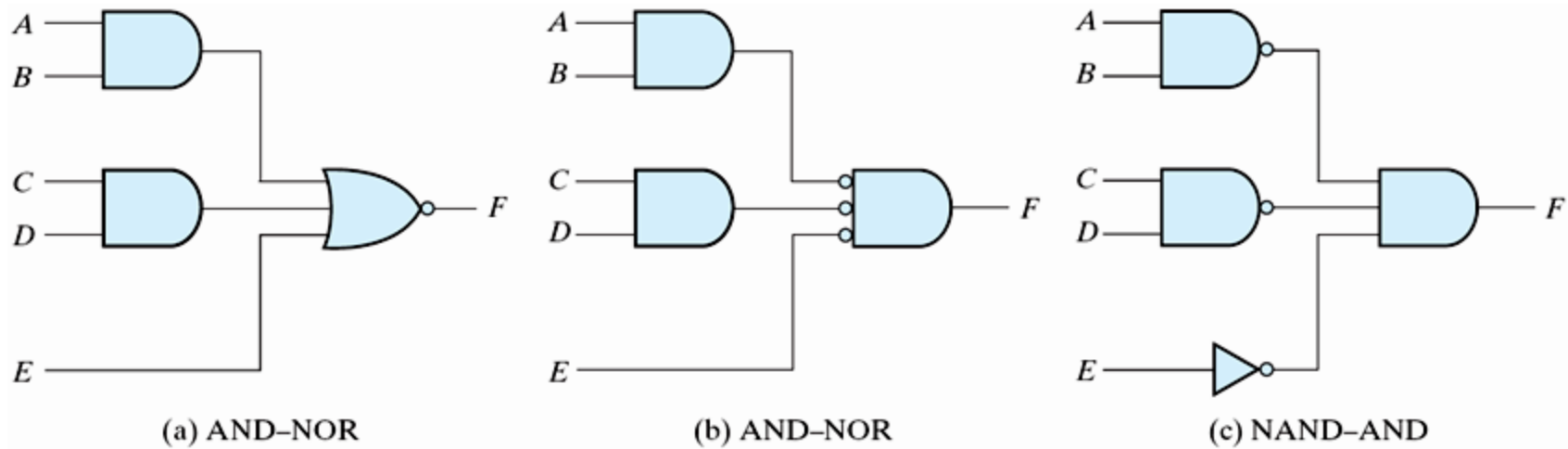
NON-DEGENERATE FORMS

- 16 possible combinations of two-level forms
- Eight of them: degenerate forms = a single operation
 - AND-AND, AND-NAND, OR-OR, OR-NOR, NAND-OR, NAND-NOR, NOR-AND, NOR-NAND.
- The eight non-degenerate forms
 - AND-OR, OR-AND, NAND-NAND, NOR-NOR, NOR-OR, NAND-AND, OR-NAND, AND-NOR.
 - **AND-OR** and **NAND-NAND** = sum of products.
 - **OR-AND** and **NOR-NOR** = product of sums.
 - **NOR-OR**, **NAND-AND**, **OR-NAND**, **AND-NOR** = ?



AND-OR-INVERT IMPLEMENTATION

- AND-OR-INVERT (AOI) Implementation
 - NAND-AND = AND-NOR = AOI
 - $F = (AB + CD + E)'$
 - $F' = AB + CD + E$ (sum of products)

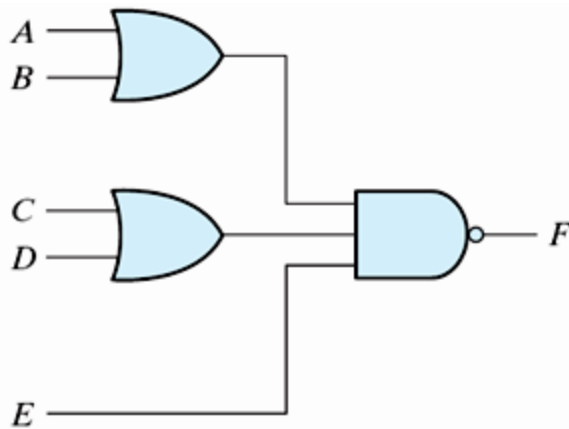


AND-OR-INVERT circuits, $F = (AB + CD + E)'$

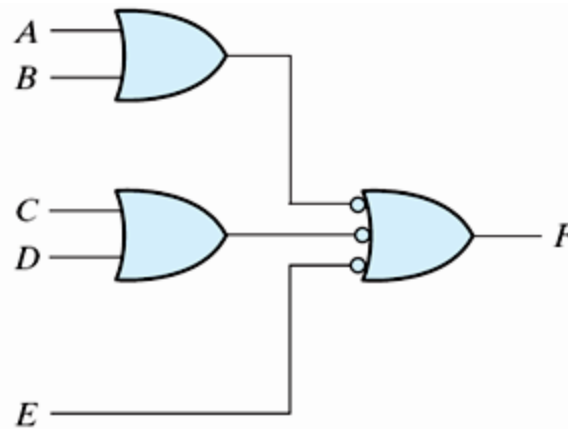


OR-AND-INVERT IMPLEMENTATION

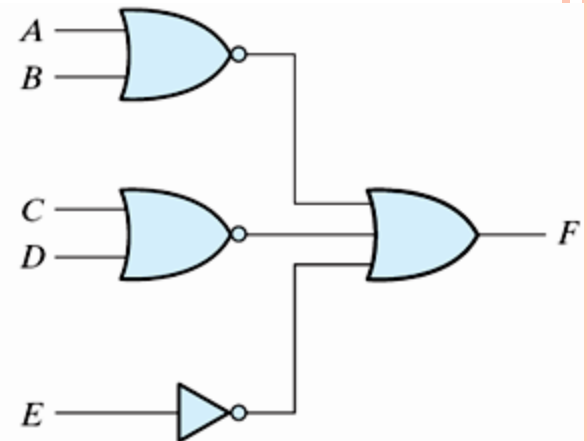
- OR-AND-INVERT (OAI) Implementation
 - OR-NAND = NOR-OR = OAI
 - $F = ((A+B)(C+D)E)'$
 - $F' = (A+B)(C+D)E$ (product of sums)



(a) OR-NAND



(b) OR-NAND



(c) NOR-OR

OR-AND-INVERT circuits, $F = ((A+B)(C+D)E)'$

TABULAR SUMMARY AND EXAMPLES

Table 3.3

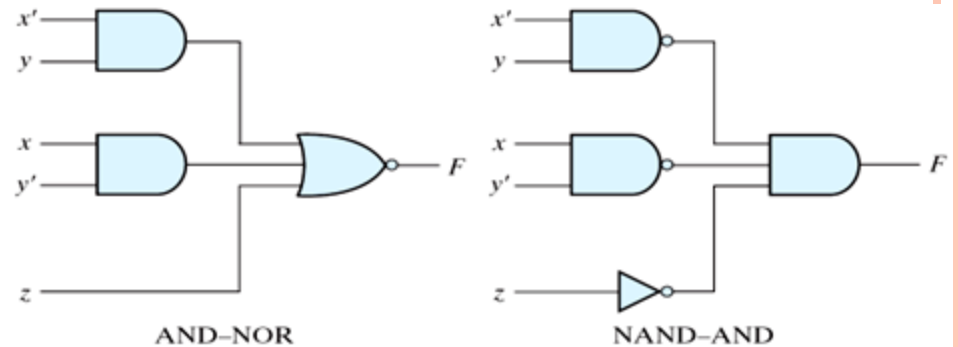
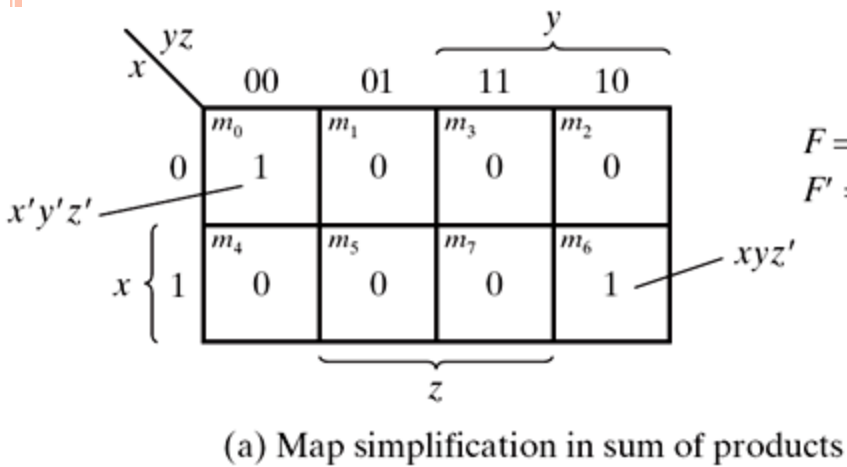
Implementation with Other Two-Level Forms

Equivalent Nondegenerate Form		Implements the Function	Simplify F' into	To Get an Output of
(a)	(b)*			
AND-NOR	NAND-AND	AND-OR-INVERT	Sum-of-products form by combining 0's in the map.	F
OR-NAND	NOR-OR	OR-AND-INVERT	Product-of-sums form by combining 1's in the map and then complementing.	F

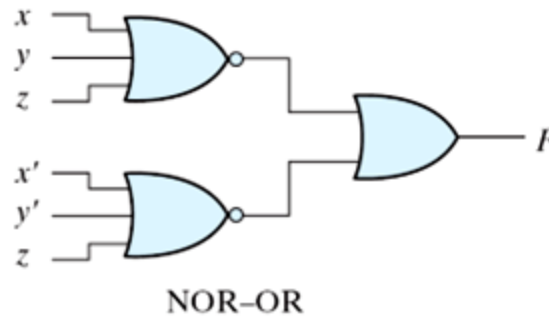
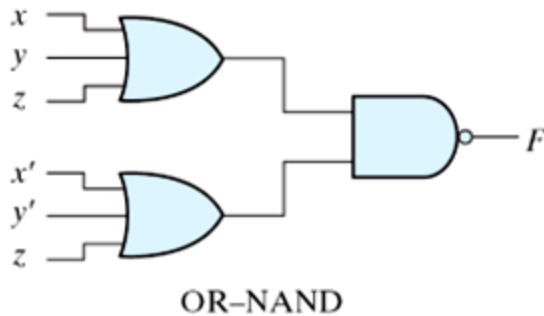
*Form (b) requires an inverter for a single literal term.



EXAMPLE



(b) $F = (x'y + xy' + z)'$



(c) $F = [(x + y + z)(x' + y' + z)]'$

Other Two-level Implementations

EXCLUSIVE-OR FUNCTION

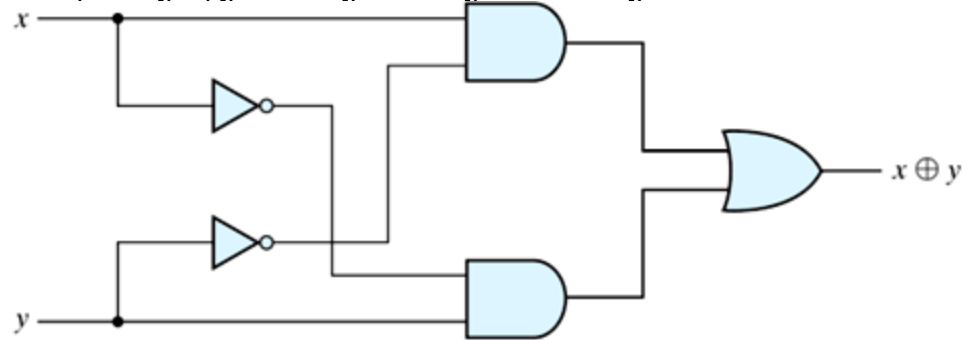
- Exclusive-OR (XOR)
 - $x \oplus y = xy' + x'y$
- Exclusive-NOR (XNOR)
 - $(x \oplus y)' = xy + x'y'$
- Some identities
 - $x \oplus 0 = x$
 - $x \oplus 1 = x'$
 - $x \oplus x = 0$
 - $x \oplus x' = 1$
 - $x \oplus y' = (x \oplus y)'$
 - $x' \oplus y = (x \oplus y)'$
- Commutative and associative
 - $A \oplus B = B \oplus A$
 - $(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$



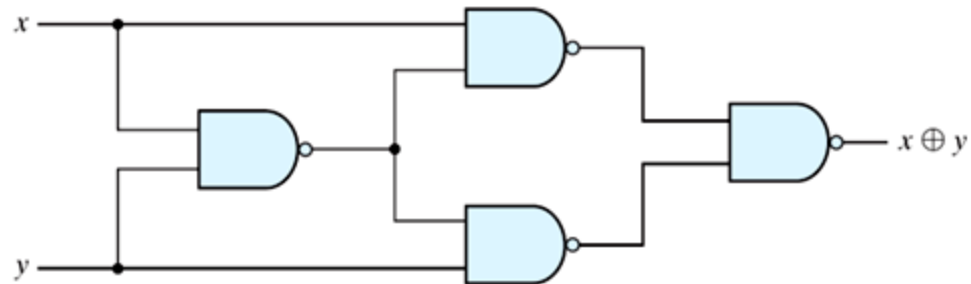
EXCLUSIVE-OR IMPLEMENTATIONS

○ Implementations

- $(x' + y')x + (x' + y')y = xy' + x'y = x \oplus y$



(a) With AND-OR-NOT gates



(b) With NAND gates

Exclusive-OR Implementations

ODD FUNCTION

- $A \oplus B \oplus C = (AB' + A'B)C' + (AB + A'B')C = AB'C' + A'BC' + ABC + A'B'C = \Sigma(1, 2, 4, 7)$
- XOR is an odd function $\rightarrow$ an odd number of 1's, then $F = 1$.

$A \backslash BC$		B			
		00	01	11	10
A	0	m_0	m_1 1	m_3	m_2 1
	1	m_4 1	m_5	m_7 1	m_6
		C			

(a) Odd function $F = A \oplus B \oplus C$

$A \backslash BC$		B			
		00	01	11	10
A	0	m_0 1	m_1	m_3 1	m_2
	1	m_4	m_5 1	m_7	m_6 1
		C			

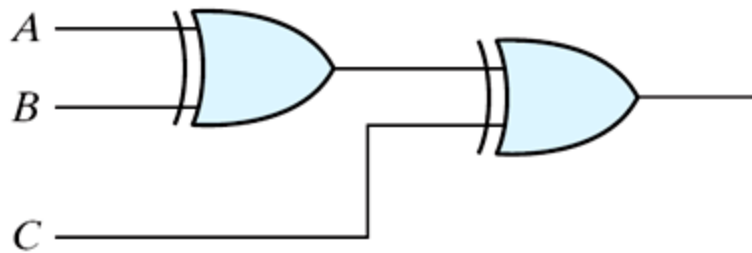
(b) Even function $F = (A \oplus B \oplus C)'$

Map for a Three-variable Exclusive-OR Function

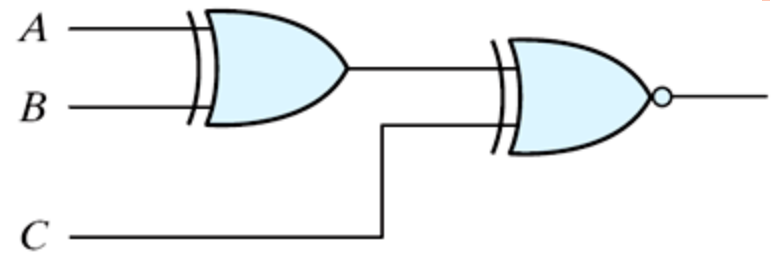


XOR AND XNOR

- Logic diagram of odd and even functions



(a) 3-input odd function



(b) 3-input even function

Logic Diagram of Odd and Even Functions



FOUR-VARIABLE EXCLUSIVE-OR FUNCTION

Four-variable Exclusive-OR function

- $$A \oplus B \oplus C \oplus D = (AB' + A'B) \oplus (CD' + C'D) = (AB' + A'B)(CD + C'D') + (AB + A'B')(CD' + C'D)$$

AB \ CD		C			
		00	01	11	10
A	00	m_0	m_1 1	m_3	m_2 1
	01	m_4 1	m_5	m_7 1	m_6
	11	m_{12}	m_{13} 1	m_{15}	m_{14} 1
	10	m_8 1	m_9	m_{11} 1	m_{10}
		D			

(a) Odd function $F = A \oplus B \oplus C \oplus D$

AB \ CD		C			
		00	01	11	10
A	00	m_0 1	m_1	m_3 1	m_2
	01	m_4	m_5 1	m_7	m_6 1
	11	m_{12} 1	m_{13}	m_{15} 1	m_{14}
	10	m_8	m_9 1	m_{11}	m_{10} 1
		D			

(b) Even function $F = (A \oplus B \oplus C \oplus D)'$

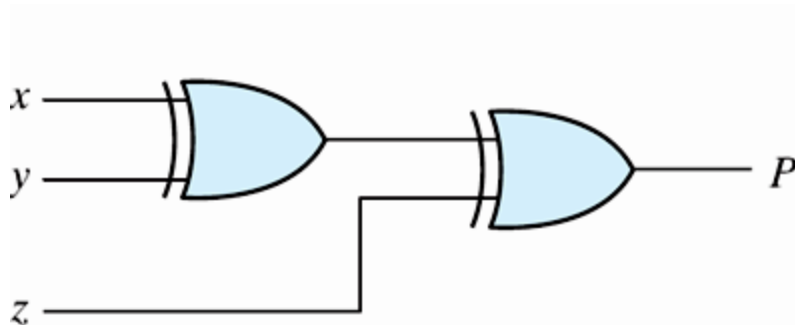
Map for a Four-variable Exclusive-OR Function



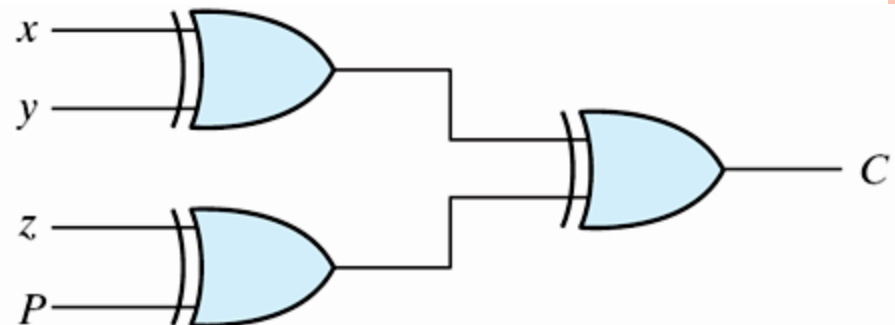
PARITY GENERATION AND CHECKING

○ Parity Generation and Checking

- A parity bit: $P = x \oplus y \oplus z$
- Parity check: $C = x \oplus y \oplus z \oplus P$
 - $C=1$: one bit error or an odd number of data bit error
 - $C=0$: correct or an even # of data bit error



(a) 3-bit even parity generator



(b) 4-bit even parity checker

Logic Diagram of a Parity Generator and Checker



PARITY GENERATION AND CHECKING

Table 3.4

Even-Parity-Generator Truth Table

Three-Bit Message			Parity Bit
<i>x</i>	<i>y</i>	<i>z</i>	<i>P</i>
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



PARITY GENERATION AND CHECKING

Table 3.5

Even-Parity-Checker Truth Table

Four Bits Received				Parity Error Check
<i>x</i>	<i>y</i>	<i>z</i>	<i>P</i>	<i>C</i>
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

